



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

INFORMÁCIÓS RENDSZEREK TANSZÉK

Asztali alkalmazás futballszurkolóknak

Témavezető:

Varga Dániel

egyetemi adjunktus, PhD

Szerző:

Lendvay Kevin Kayasit

programtervező informatikus BSc

Budapest, 2025

SZAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Lendvay Kevin Kayasit

Neptun kód: AJZ5CT

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Varga Dániel

munkahelyének neve, tanszéke: ELTE IK, Információs rendszerek Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: egyetemi adjunktus, doktori fokozat

A szakdolgozat címe: Asztali alkalmazás futballszurkolóknak

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

A szakdolgozat célja egy Python és PyQt alapú asztali alkalmazás fejlesztése, amely a futballmérkőzések eredményeit próbálja előre jelezni különböző statisztikai és gépi tanulási modellek segítségével. Az alkalmazás célcsoportja a futballrajongók, akik szeretnék statisztikai adatokra alapozva elemezni a mérkőzéseket és jobb döntéseket hozni, például fogadások során. A rendszer különböző elemzési lehetőségeket biztosít, segítve a felhasználókat abban, hogy megalapozott döntéseket hozzanak, akár sportfogadás, akár a csapatok teljesítményének megértése terén.

Az alkalmazás különféle statisztikai adatokat jelenít meg, beleértve a csapatok, játékosok és a fogadóirodák részletes statisztikáit. A felhasználók betekintést nyerhetnek a csapatok előző mérkőzéseinek teljesítményébe, a játékosok statisztikai mutatóiba (pl. gólok, asszisztok, sérülések) és a különböző fogadóirodák által kínált esélyekbe. Ezen adatok segítségével a felhasználók alaposabb elemzéseket végezhetnek és könnyebben megérthetik a mérkőzések várható kimenetelét.

A fejlesztés során az alkalmazás egy adatbázisban tárolja a statisztikai adatokat és elemzéseket, amelyeket a felületen is megjelenít ábrák formájában. Ezen kívül az alkalmazás folyamatosan frissíti az adatokat API-k segítségével, így a mérkőzésekkel kapcsolatos legfrissebb információk, például a fogadóirodák esélyei, az aktuális csapatteljesítmények és a ligák tabellái mindig elérhetőek lesznek.

A felhasználók számára a rendszer különböző modelleket kínál, amelyek segítségével előrejelzéseket készíthetnek a mérkőzések eredményeiről. Az alkalmazás lehetőséget biztosít a modellek összehasonlítására, így a felhasználók többféle előrejelzést is megvizsgálhatnak, és azok alapján hozhatják meg döntéseiket. A mérkőzések áttekintését egy naptárstruktúra segíti, amely lehetővé teszi, hogy a felhasználók könnyedén nyomon követhessék a közelgő eseményeket, és ne hagyjanak ki egyetlen fontos mérkőzést sem.

Az alkalmazás egy minigame funkciót is tartalmaz, amelyben a felhasználók pontokat gyűjthetnek a játék során. A felhasználók személyre szabott felhasználói fiókot hozhatnak létre, amelyben követhetik pontjaikat és eredményeiket. A pontok egy ranglista segítségével nyomon követhetők, így a felhasználók láthatják, hogyan teljesítenek másokkal szemben.

Összességében az alkalmazás célja, hogy egy átlátható és könnyen kezelhető eszközt biztosítson a futballrajongók számára, amely segít jobban megérteni és előre jelezni a mérkőzések eredményeit, valamint lehetőséget ad a pontosabb elemzésekre és döntésekre.

Budapest, 2025. 03. 13.

SZAKDOLGOZAT / DIPLOMAMUNKA

EREDETISÉG NYILATKOZAT

Alulírott Lendvay Kevin Károl Neptun-kód: AJZ5CT

ezennel kijelentem és aláírással megerősítem, hogy az Eötvös Loránd Tudományegyetem

Informatikai Karának, Információs Rendszerek Tanszékén írt,

..... Arztali alkalmazás futballeszmozgások

című szakdolgozatom/diplomamunkám saját, önálló szellemi termékem; az abban hivatkozott szakirodalom felhasználása a szerzői jogok általános szabályainak megfelelően történt.

Tudomásul veszem, hogy szakdolgozat/diplomamunka esetén plágiumnak számít:

- szó szerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Budapest, 2025.11.23

Lendvay Kevin

.....
hallgató aláírása

Tartalomjegyzék

1. Bevezetés	4
2. Felhasználói dokumentáció	5
2.1. A futball szabályrendszere és modern elemzési módszerei	5
2.1.1. A labdarúgás rövid áttekintése	5
2.1.2. A modern futballelemzés és az xG mutató szerepe	6
2.2. Az alkalmazás rövid ismertetése	7
2.3. Rendszerkövetelmények	7
2.4. Szoftvertelepítés	8
2.5. Az alkalmazás indítása	9
2.6. Előrejelzés készítése	10
2.7. Statisztikák megtekintése	14
2.8. Mini-játékok	17
2.8.1. Bejelentkezés	18
2.8.2. Regisztráció	18
2.8.3. Mini-játék gyűjtemény	19
2.8.4. FUT csomagnyitó	20
2.8.5. Kockázat játék	21
2.8.6. Kígyó játék	22
2.8.7. Aknakereső	24
2.8.8. Cselezőkirály	26
2.8.9. Reflex játék	26
2.9. Új modell tanítása	27
2.10. Visszajelzések	29
3. Fejlesztői dokumentáció	31
3.1. Követelmény elemzés	31
3.1.1. Funkcionális követelmények	31

3.1.2. Nem funkcionális követelmények	33
3.2. Fejlesztői környezet és technológiák	33
3.3. A rendszer architektúrális felépítése	35
3.4. Az alkalmazás belépési pontja	37
3.5. Perzisztencia	40
3.5.1. Predikció adatbázis	40
3.5.2. Mini-játék adatbázis	41
3.5.3. Külső API-k	43
3.6. A modell réteg	44
3.6.1. Predikciós modell	44
3.6.2. Statisztika modell	47
3.6.3. Autentikáció	47
3.6.4. Mini-játék modellek	48
3.6.5. Tanító logika	50
3.7. Vezérlő réteg	51
3.7.1. Az előrejelzések irányítása	52
3.7.2. A statisztikák vezérlése	52
3.7.3. A TrainingController osztály	53
3.7.4. A mini-játékokat irányító osztályok	53
3.8. Felhasználói felület	56
3.9. Gépi tanulásos modellek tanítása	58
3.9.1. Adathalmaz	58
3.9.2. Tanítás	59
3.10. Tesztelés	62
3.10.1. Egységtesztelés	62
3.10.2. Manuális tesztelés	65
3.11. Továbbfejlesztési lehetőségek	68
4. Összegzés	70
Köszönetnyilvánítás	71
A. Tanítási eredmények	72
B. Hiperparaméter finomhangolás	73

B.1. A finomhangolt hiperparaméterek	73
B.1.1. XGBoost	73
B.1.2. Gradient Boosting Regressor	74
B.1.3. Multi-Layer Perceptron	74
C. A mesterséges intelligencia használatának nyilatkozata	75
Irodalomjegyzék	76
Ábrajegyzék	78
Táblázatjegyzék	80
Forráskódjegyzék	81

1. fejezet

Bevezetés

Az elmúlt években a mesterséges intelligencia (MI) rendkívül gyors ütemben fejlődött, és mára az élet számos területén meghatározó szerepet tölt be – az ipari automatizálástól egészen a mindennapi felhasználási területekig. Az MI egyik legfontosabb ága, a gépi tanulás, új távlatokat nyitott a fejlesztők és kutatók számára, mivel lehetővé teszi mintázatok és összefüggések felismerését nagyméretű adathalmazokból. Ez a technológia egyre inkább a modern szoftverfejlesztés szerves részévé válik, ugyanakkor alkalmazása tudatos és felelősségteljes megközelítést igényel.

A mesterséges intelligencia fejlődése számomra is inspiráló hatású volt, noha kezdetben inkább távolról figyeltem ezt a területet. Az informatika iránti érdeklődésem elmélyülésével azonban a gépi tanulás iránt is fokozatosan nőtt a kíváncsiságom. Mivel a futball szintén meghatározó szerepet játszik az életemben, természetes módon merült fel bennem a gondolat, hogy e két területet – az informatikát és a sportot – összekapcsoljam.

A szakdolgozat célja egy Python alapú asztali alkalmazás fejlesztése, amely gépi tanulási modellek segítségével egy futballhoz kapcsolódó problémát vizsgál és old meg. A projekt középpontjában a megfelelő modellek kiválasztása, az adatok elemzése, valamint az eredmények vizuális megjelenítése áll, hogy a végeredmény egy hasznos, adatvezérelt eszköz legyen a futballrajongók és elemzők számára.

2. fejezet

Felhasználói dokumentáció

2.1. A futball szabályrendszere és modern elemzési módszerei

2.1.1. A labdarúgás rövid áttekintése

Bár a labdarúgás a világ egyik legelterjedtebb sportja, nem mindenki követi figyelemmel. Az alkalmazás központi eleme egy futballprobléma megoldása, ezért az alapvető ismeretek elengedhetetlenek a működés teljes megértéséhez. A játék eredete egészen az ókorig visszanyúlik; azonban modern, szabványosított formája a 19. századi Angliában alakult ki. Kezdetben középiskolák és felsőoktatási intézmények mérköztek egymással, később pedig profi magánklubok bevonásával létrejöttek az első nagyobb bajnokságok is. A sportág gyors elterjedését segítette, hogy fizikai adottságtól függetlenül szinte bárki játszhatta, így hamar népszerűvé vált egész Európában.

A játékot két, egyenként tizenegy fős csapat játssza egymás ellen. A cél az, hogy a labdát a mérkőzés során minél több alkalommal az ellenfél kapujába juttassák. A labda kezelése bármely testrészrel engedélyezett, kivéve az alkar és a felkar használatát; ez alól a kapus jelent kivételt a saját tizenhatosán belül. A modern futball szabályainak kialakítása és karbantartása a Nemzetközi Futbalszövetség Testületének (IFAB) feladata, amely évről évre frissíti a Laws of the Game dokumentumot. A sport globális irányítását azonban a FIFA¹ végzi, amely a világversenyek, így a

¹<https://www.fifa.com/en>

labdarúgó-világbajnokság szervezéséért, valamint a sportág nemzetközi szintű működtetéséért felel.

A szabályok mára rendkívül összetettek és folyamatosan fejlődnek, ezért a pontos értelmezéshez és további részletek megismeréséhez ajánlott az IFAB hivatalos oldalán elérhető Laws of the Game [1] tanulmányozása, amely a legfrissebb és legmegbízhatóbb iránymutatásokat tartalmazza.

2.1.2. A modern futballelemzés és az xG mutató szerepe

A labdarúgás elemzése közel ötven évre nyúlik vissza, egészen Charles Reep munkásságáig. Reep, aki elkötelezett futballrajongó volt, csapata eredménytelensége nyomán kezdett a problémák okait keresni. Ő volt az első, aki rendszeresen jegyzetelte a mérkőzések statisztikáit, és próbált következtetéseket levonni belőlük. Reep egy egyszerű, ám annál meghatározóbb jelenséget figyelt meg: a gólok többsége kevés passz után születik, így a sok passzos játék kevésbé hatékony. E megfigyelés forradalmasította az angol labdarúgást, és az angol kluboknál a mai napig a gyors, kevés passzos játék a legelterjedtebb. Ez a megállapítás a modern futballban is helytálló, mivel a masszív védekezést leginkább gyors és kreatív játékmegoldásokkal lehet feltörni.

A modern labdarúgás kezdetének gyakran az 1992-es évet tekintik, amikor megalakult az angol Premier League, és a Bajnokcsapatok Európa Kupája (BEK) átalakult a Bajnokok Ligájává. 1994-ben az OPTA² vállalat kezdett el részletes adatgyűjtést az angol bajnokságról, hogy pontosabb erőviszonyokat tudjon felállítani. Korábban soha nem látott részletességgel jegyezték fel adatokat, amelyek felkeltették más cégek és klubok érdeklődését is. Az OPTA tevékenysége jelentette az áttörést az adatgyűjtés terén, és a 1990-es évektől kezdve hatalmas mennyiségű információ halmozódott fel a világ különböző bajnokságaiból.

Egy labdarúgó-mérkőzés előrejelzése rendkívül összetett feladat, mivel számos paraméter figyelembevételére van szükség. Ennek megoldására a modern statisztikai elemzések nem egyetlen modellen, hanem mintahalmazon alapuló megközelítéssel dolgoznak. Az adatgyűjtés 2010-re érte el azt a szintet, hogy elegendő mintapélda álljon rendelkezésre olyan modellek betanításához, amelyek képesek megbecsülni a várható gólok számát (expected goals, röviden xG). A további statisztikák és mo-

²https://optaplayerstats.statsperform.com/en_GB/soccer

dellek, amelyeket a mai labdarúgásban alkalmaznak, részletesen elérhetők az xfb Analytics [2] oldalon.

2.2. Az alkalmazás rövid ismertetése

Az xGoalPro egy olyan asztali alkalmazás, amely a futballrajongók és sportfogadók számára biztosít elemző és döntéstámogató felületet. Az alkalmazás célja, hogy gépi tanulási modellek segítségével előrejelezze a futballmérkőzések várható kimenetelét, és az eredményeket vizuálisan is megjelenítse. A statisztikai adatok és grafikus ábrák segítségével a felhasználók részletes elemzéseket végezhetnek, ezáltal megalapozottabb döntéseket hozhatnak sportfogadásaik során.

Az alkalmazás adatforrásai automatikusan frissülnek, így a felhasználók mindig naprakész információkat kapnak a csapatok, játékosok és bajnokságok aktuális teljesítményéről. A rendszer lehetőséget kínál egyedi gépi tanulási modellek betanítására is, amennyiben a beépített modellek nem érik el a kívánt pontosságot.

Az xGoalPro további funkciói közé tartoznak a beépített mini-játékok, amelyek a felhasználói élményt hivatottak növelni, és kikapcsolódást biztosítanak a mérkőzések közötti időszakban.

2.3. Rendszerkövetelmények

Az alábbiakban találhatók az alkalmazás telepítéséhez és futtatásához szükséges csomagok és rendszererőforrások.

- Microsoft .NET 7.0 (x64)³
- legalább 1 GB RAM
- legalább 300 Mb
- tanításhoz: 8 GB RAM
- legalább Windows 10

Az alkalmazás fejlesztését és tesztelését a saját számítógémemen végeztem: a fejlesztést laptopon, míg a tanítást és tesztelést asztali számítógépen (PC) hajtottam végre. Az eszközök specifikációi a következők:

³<https://dotnet.microsoft.com/en-us/download/dotnet/7.0/runtime>

1. Asztali számítógép:

- Windows 11 Home, 64 bites operációs rendszer
- Inter Core i7-10700
- 16 GB memória

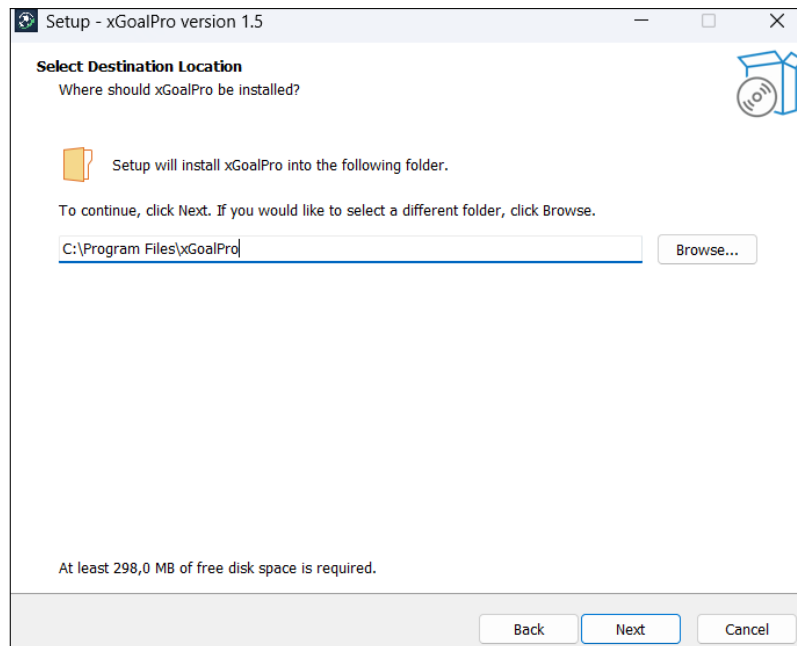
2. Laptop:

- Windows 11 Home, 64 bites operációs rendszer
- Inter Core i5-1135G7
- 8 GB memória

2.4. Szoftvertelepítés

Az alkalmazáshoz elkészítettem egy telepítőprogramot. A rendszerkövetelmények figyelembevételével a felhasználó a telepítést egyszerűen elindíthatja: a fájlt le kell tölteni a megosztott helyről, majd futtatni. A folyamat megkezdéséhez rendszergazdai jogosultság szükséges. Ezt követően a felhasználónak lehetősége van áttekinteni a felhasználási feltételeket, amelyeket el kell fogadnia a továbblépéshez.

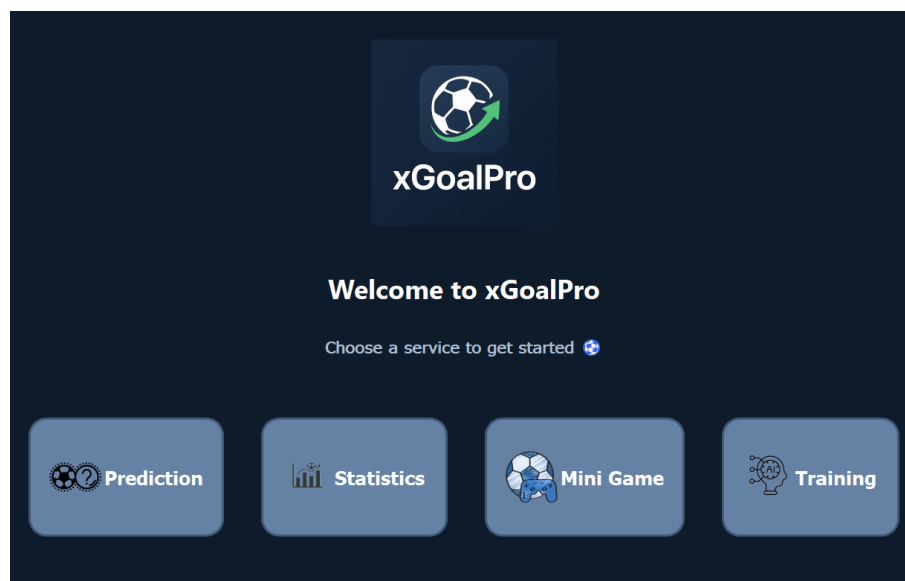
A telepítő biztosítja a program célmappájának kiválasztását, valamint az asztali parancsikon létrehozását a felhasználó igényei szerint. A varázsló utolsó lépésében a telepítés elindul, és a program a megadott helyre kerül. A telepített alkalmazás mappájában található egy eltávolító program is, amely a szoftver egyszerű törlését teszi lehetővé. A telepítés befejeztével az alkalmazás a mappában található indítófájltra kattintva azonnal használatba vehető.



2.1. ábra. xGoalPro telepítés

2.5. Az alkalmazás indítása

A program indítása után a kezdőoldal tölt be. A kezdőoldalon látható (2.2 ábra) az alkalmazás logója és egy rövid köszöntő szöveg, amely bevezeti a felhasználót az alkalmazás használatába. A logó és a szöveg alatt található négy gomb, amelyekkel a négy fő szolgáltatás közül lehet választani.



2.2. ábra. xGoalPro főoldal

Predikció (angolul: Prediction) opcióban gépi tanulmányos modellek segítségével lehet előrejelezni labdarúgó mérkőzések várható kimenetelét.

Statisztika (angolul: Statistics) menüben a futballcsapatok és játékosok statisztikai tekinthetők meg, továbbá a korábban készített előrejelzések eredményei is visszanézhetők.

Mini-játék (angolul: Minigame) gombra kattintva érhetők el a kikapcsolódásra szánt kisebb játékok. A mini-játékok rövid, egyszerűen teljesíthető feladatokat tartalmaznak, például futball-tematikájú ügyességi játékokat. Ezek célja a felhasználói élmény javítása és egy könnyed szünet biztosítása az alkalmazáson belül.

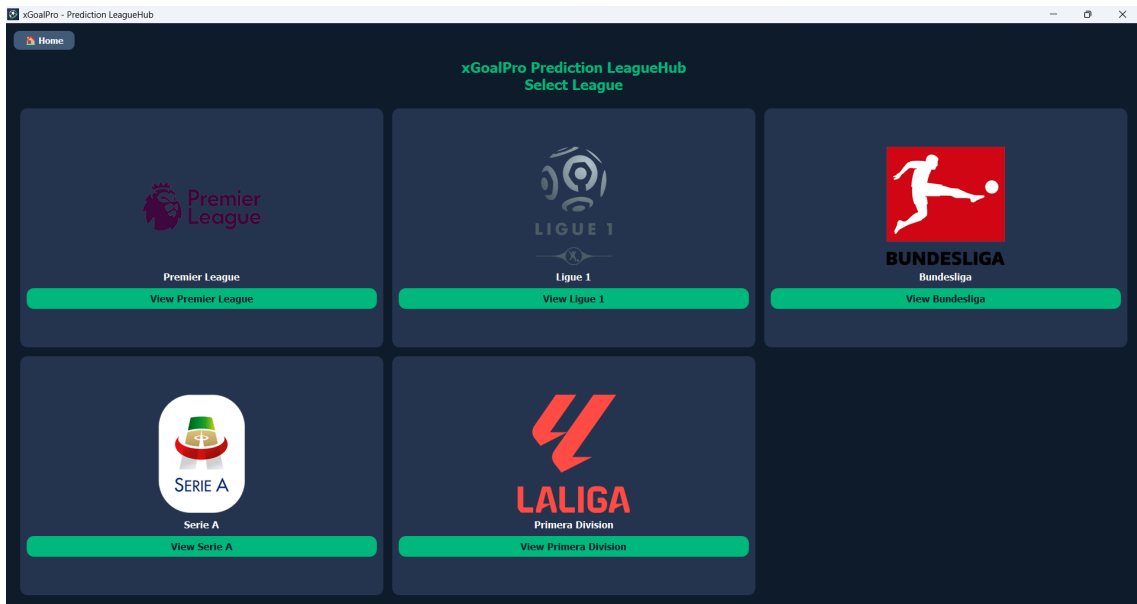
Tanítás (angolul: Training) menü egy haladóbb funkciót ad az alkalmazáshoz, ahol saját modellt lehet betanítani különféle paraméterek megadásával.

Az egyes menüpontok részletes leírását és használatuk ismertetését az alább, a menüpontok nevéhez kapcsolódó szekciókban találhatjuk.

2.6. Előrejelzés készítése

A predikció elkészítésének első lépése a bajnokság kiválasztása. A felhasználó a főoldalon (2.2 ábra) található „Predikció” gomb megnyomásával a liga választó felületre (2.3 ábra) navigál. A főoldalra történő visszalépés a bal felső sarokban elhelyezett navigációs elem segítségével lehetséges.

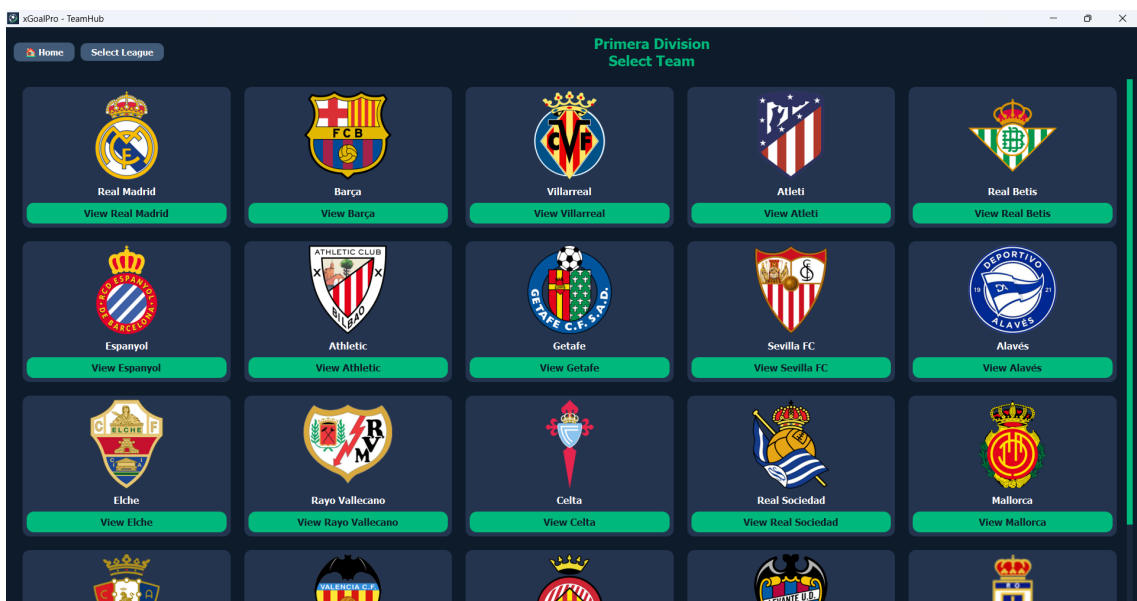
A ligák kiválasztását dinamikus „liga kártyák” segítik, amelyek tartalmazzák a bajnokság logóját, nevét és egy gombot, amely a liga kiválasztását teszi lehetővé. Jelenleg a felhasználó a világ öt legerősebb labdarúgó-bajnoksága közül választhat, azonban a rendszer kialakítása lehetővé teszi a jövőbeni bővítést további ligákkal.



2.3. ábra. Liga választás

A bajnokság kiválasztását követően a rendszer a csapatválasztó oldalt (2.4 ábra) tölti be, amely az adott liga aktuális idényének csapatait jeleníti meg. Hasonlóan a liga választó felülethez (2.3 ábra), a navigáció a bal felső sarokban található, lehetővé téve a visszatérést a főoldalra vagy az előző oldalra.

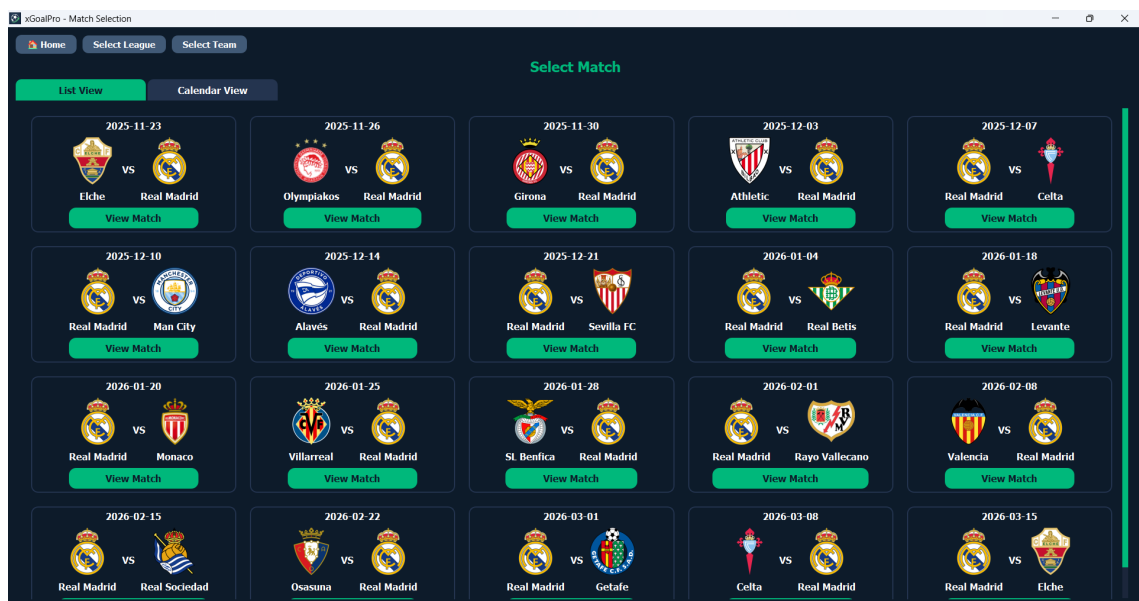
Minden csapat egy dinamikus „csapatkártyán” jelenik meg, amely tartalmazza a csapat címerét, nevét, valamint egy gombot, amely segítségével a felhasználó a kívánt csapatot választhatja ki.



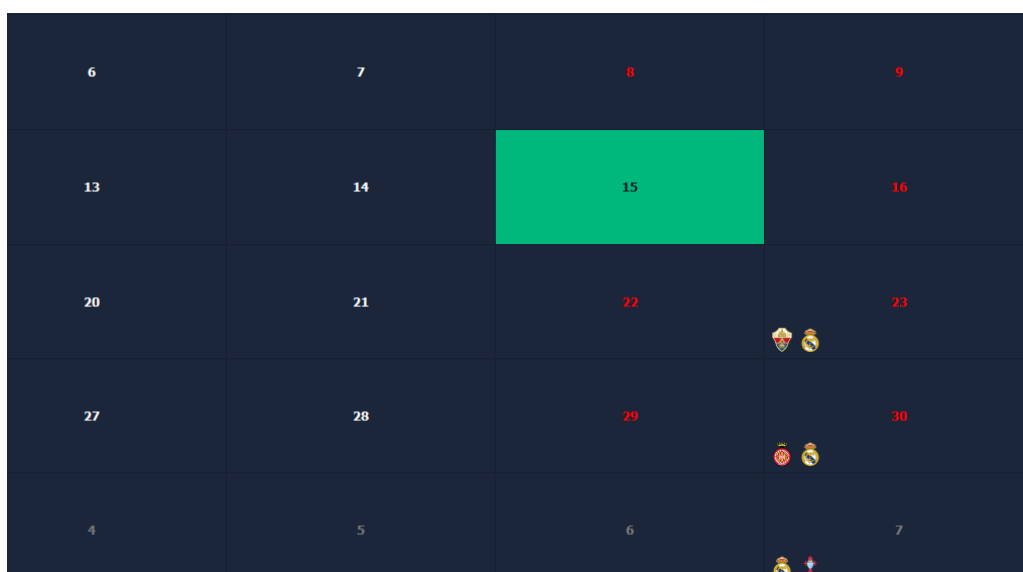
2.4. ábra. Csapat választás előrejelzéshez

A csapat kiválasztása után betöltődik a mérkőzésválasztó oldal (2.5 ábra), ahol az aktuális csapat következő 20 mérkőzését láthatjuk. A navigáció a korábbi ablakokhoz a bal felső sarokban található. A felületen két nézet közül választhatunk a fenti fül navigáció segítségével.

- Lista nézet (2.5 ábra): A mérkőzések kártyákon jelennek meg, és az alattuk található gomb segítségével választhatjuk ki azokat.
- Naptár nézet (2.6 ábra): A mérkőzések egy hagyományos naptárban láthatók, és az adott naptári cellára kattintva tudjuk kiválasztani őket.



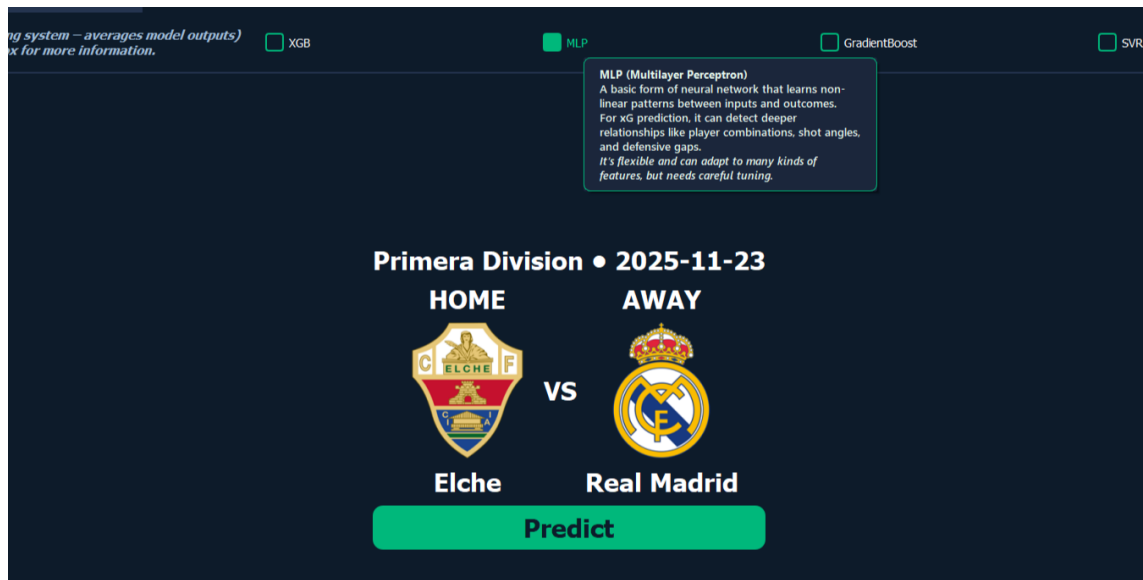
2.5. ábra. Mérkőzés választás előrejelzéshez



2.6. ábra. Mérkőzések naptár nézetben

A mérkőzés kiválasztása után betöltődik az oldal (2.7 ábra), ahol előrejelzést készíthetünk. A bal felső sarokban található a navigáció a korábbi ablakokhoz.

Középen látható a mérkőzés „kártyája” (2.7 ábra), alatta pedig egy gomb, amivel megkezdhetjük a predikciót. A predikcióhoz a felső jelölőnégyzet-sorban kell kiválasztani a használni kívánt modell(ek)e)t. Ha az egérrel egy jelölőnégyzet fölé visszük a kurzort, az adott modellelről részletes információkat kapunk.



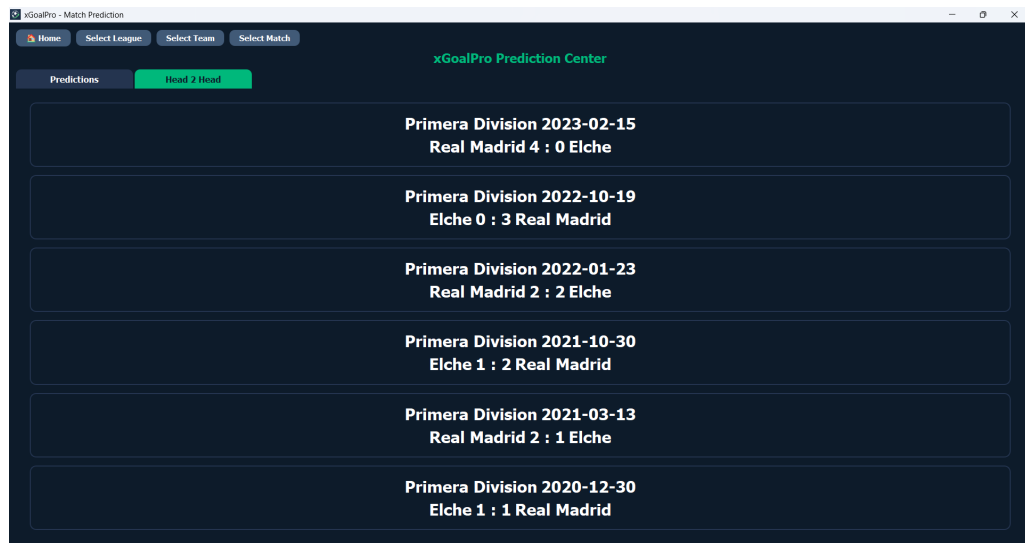
2.7. ábra. Előrejelző ablak részlet

A „Predikció” gomb megnyomásával elindul az előrejelzési folyamat, és néhány másodperc elteltével az oldal frissül, az alsó részen megjelenítve az eredményeket (2.8 ábra).

Elche	Real Madrid
Predict	
PREDICTION RESULT	
Expected Home Goals: 0.85	
Expected Away Goals: 1.63	
P Homewin (%): 19.45	
P Draw (%): 24.70	
P Awaywin (%): 55.85	

2.8. ábra. Előrejelzés eredmény

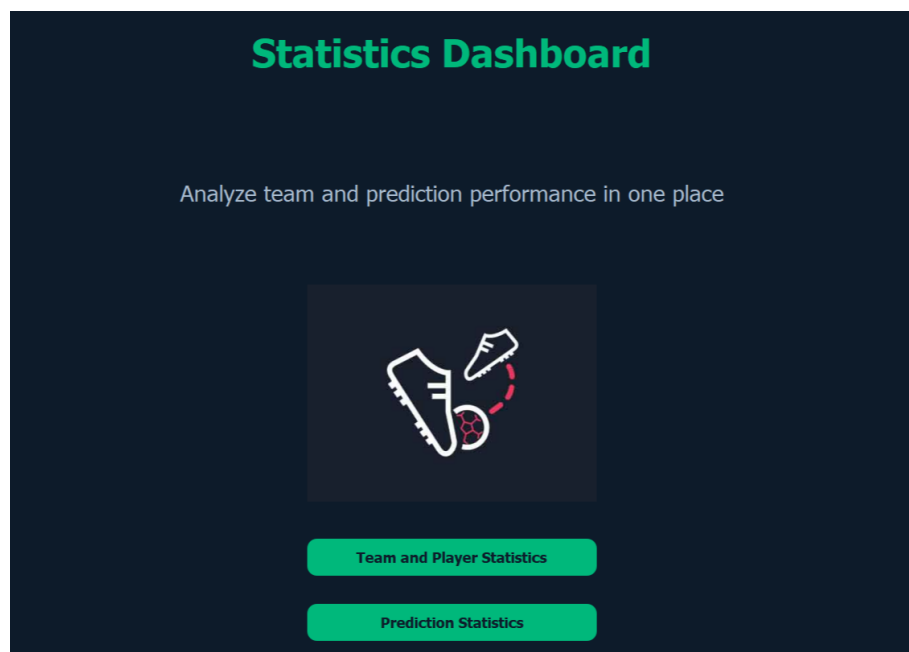
A fül navigáció segítségével megtekinthetjük a két csapat egymás elleni mérkőzéseit (2.9 ábra) akár öt évre visszamenőleg.



2.9. ábra. Egymás elleni mérkőzések

2.7. Statisztikák megtekintése

A főoldalon (2.2 ábra) a Statisztika gomb lenyomásával betölt a statisztika ablak kezdőoldala (2.10 ábra). Az üdvözlő szöveg és egy kép alatt két gomb található, amelyekkel választhatunk a predikciós adatok és a csapatadatok megtekintése között. A bal felső sarokban található a navigáció az alkalmazás főoldalára.



2.10. ábra. Statisztika kezdőoldal

A predikciós statisztikára kattintva a statisztika főoldalon (2.10 ábra) egy új ablak tölt be (2.11 ábra), ahol megtekinthetjük az előrejelzésekből készült diagramokat. Jelenleg a következő diagramok érhetők el:

- Várható gólok eloszlása
- Modellek pontossága
- Valószínűségek eloszlása
- Modellek eredményeinek átlaga

A korábbi ablakokhoz hasonlóan a bal felső sarokban található a főoldalra és a statisztika kezdőoldalára visszavezető gombok.



2.11. ábra. Predikció ábrák

A fül navigációval átváltva táblázatos formában (2.12 ábra) is láthatjuk az összes predikciót. A táblázatban található egy kód oszlop, amely jelzi, hogy az egyes modellek fel voltak-e használva az adott előrejelzés során (0 – a modell nem lett kiválasztva, 1 – a modell ki lett választva). Ha az egeret egy cella fölé visszük, egy buborékszerűen jelennek meg a cella tartalmára vonatkozó információk.



Match	Model	Home xG	Away xG	Home Win %
Getafe vs Real Madrid	1111	0.88	1.90	16.8%
Fulham vs Arsenal	0110	0.97	1.30	27.8%
Bayern vs Dortmund	1010	2.31	1.04	65.9%
Torino vs Napoli	0111	1.00	1.38	27.3%
PSG vs Strasbourg	1000	2.63	0.95	73.2%
Liverpool vs Man United	1010	2.59	0.95	72.7%
Brentford vs Liverpool	0100	1.12	1.51	28.1%
Villarreal vs Real Betis	1010	1.29	1.07	41.6%
Crystal Palace vs Bournemouth	0100	1.56	1.02	49.8%
Bournemouth vs Nottingham	0110	1.68	0.95	54.6%
Cheltenham vs Middlesbrough	1010	1.30	1.01	43.0%

2.12. ábra. Előrejelzések táblázat részlet

A statisztika főoldalon (2.10 ábra) a Csapat és Játékos gombra kattintva betölt egy hasonló ligaválasztó oldal (2.3 ábra), mint ami a predikciós menüben belül látható volt. Itt kiválaszthatjuk, hogy melyik bajnokság adatait szeretnénk megtekinteni. A korábbi oldalakhoz való navigáció továbbra is a bal felső sarokban található.

Egy bajnokság adatainak kiválasztása után az információk egy új ablakban töltődnek be (2.13 ábra). A korábbi oldalakhoz való visszanavigálás továbbra is a bal felső sarokban található. A felső sáv alatt három fül közül választhatunk, attól függően, hogy mely statisztikát szeretnénk megtekinteni:

- Tabella nézetben láthatjuk az aktuális rangsort.
- Góllövő lista nézetben a legtöbb gólt szerző játékosok statisztikai érhetők el.
- Eredmények nézetben a bajnokság összes mérkőzésének végeredményei láthatók.

Ha az egérrel egy oszlop fölé visszük a kurzort, egy buborékszerű jelenség jelenik meg további információval. Egy csapat adatait a Tabella nézetben a klub nevére kattintva tekinthetjük meg.

League Table

Top Scorers

Results

Tip: Click on a team name

Hover over a column header

Team	M	W	D
Real Madrid	13	10	2
Barça	13	10	1
Villarreal	13	9	2
Atleti	13	8	4
Real Betis	13	5	6
Espanyol	13	6	3
Getafe	13	5	2
Athletic	13	5	2
Real Sociedad	13	4	4

2.13. ábra. Bajnokság statisztikák részlet

A csapat kiválasztása után egy új ablakban töltődik be az adott csapat statisztikája (2.14 ábra). A korábbi oldalakhoz való navigáció a bal felső sarokban található. A bajnokság ablakhoz hasonlóan három fül közül választhatunk, attól függően, hogy mely adatokat szeretnénk megtekinteni:

- Játékos nézetben a klub összes játékosa látható.
- Eredmények nézetben a csapat összes mérkőzésének végeredménye látható.

Players		
Results		
Arsenal FC (England) Founded: 1886 Stadium: Emirates Stadium http://www.arsenal.com		
Name	Position	Date of Birth
Kepa Arrizabalaga	Goalkeeper	1994-10-03
David Raya	Goalkeeper	1995-09-15
Tommy Setford	Goalkeeper	2006-03-13
Aleksei Fedorushchenko	Goalkeeper	2005-09-28
Max Dowman	Midfield	2009-12-31
Andre Harriman-Annous	Offence	2007-12-27
Myles Lewis-Skelly	Left-Back	2006-09-26
Ethan Nwaneri	Attacking Midfield	2007-03-21
Cristhian Mosquera	Centre-Back	2004-06-27

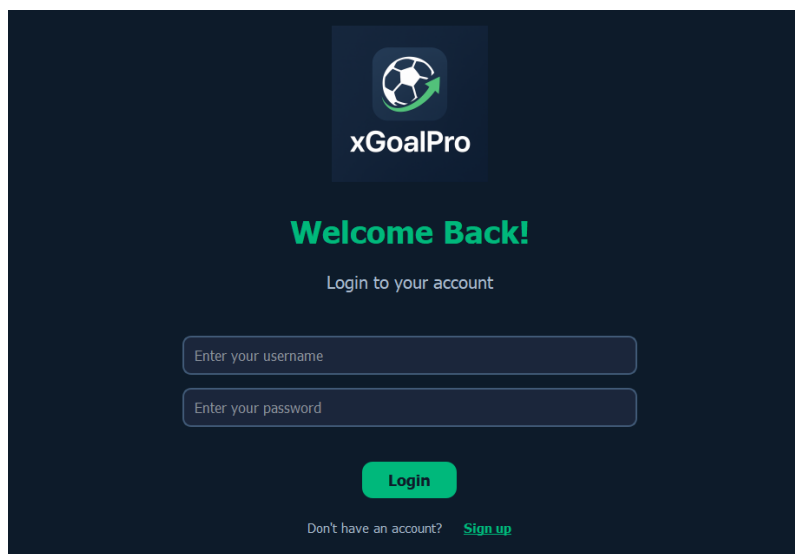
2.14. ábra. Csapat statisztikák részlet

2.8. Mini-játékok

Az alkalmazás a felhasználók számára lehetőséget biztosít a könnyed kikapcsolódásra mini-játékok szolgáltatás alatt.

2.8.1. Bejelentkezés

A főoldalon (2.2 ábra) található „Mini-játékok” gombra kattintva a felhasználó a bejelentkező oldalra (2.15 ábra) kerül. A játékok eléréséhez érvényes felhasználói fiókkal kell rendelkezni. Amennyiben a felhasználó még nem hozott létre fiókot, vagy új fiókot szeretne regisztrálni, a regisztrációs link segítségével a megfelelő oldalra navigálhat, ahol létrehozhatja saját fiókját. A felhasználó a bejelentkezési mezők kitöltését követően a „Bejelentkezés” (angolul: „Login”) gomb megnyomásával léphet be fiókjába.



2.15. ábra. Bejelentkezés

2.8.2. Regisztráció

A regisztrációs oldalra (2.16 ábra) navigálva az űrlap mezőiben adhatjuk meg az adatokat az új felhasználói fiók létrehozásához. A felhasználónév és jelszó követelményeit a (Táblázat 2.1) táblázat részletezi. Az űrlap alján található gomb segítségével indítható el a regisztráció folyamata. Sikeres regisztrációt követően az alkalmazás automatikusan átirányítja a felhasználót a bejelentkező oldalra. A bejelentkező oldalra emellett az alsó navigációs link segítségével is visszatérhetünk.

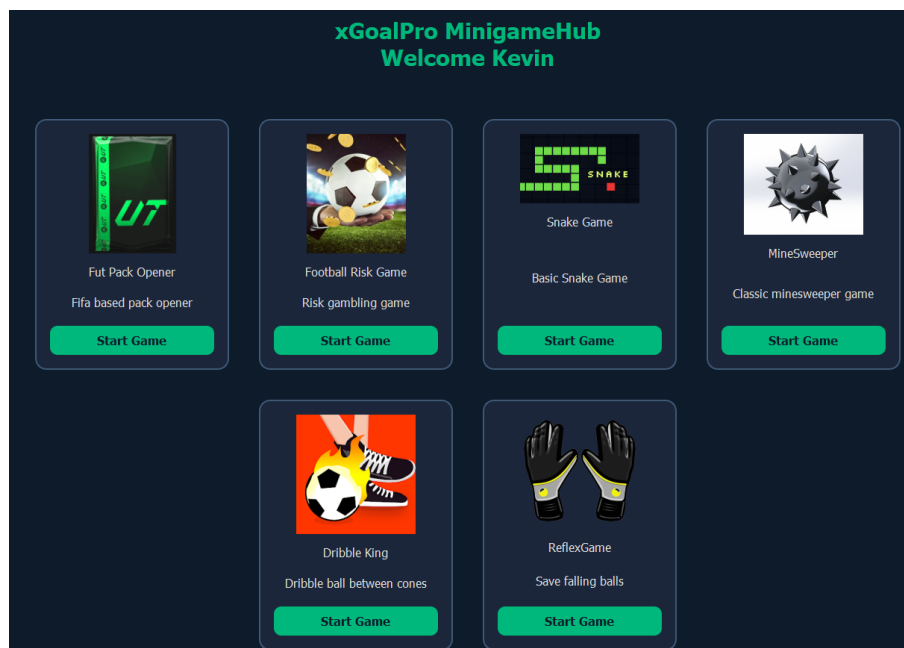
Mező	Követelmény
<i>Felhasználónév</i>	
	Legalább 3 karakter hosszú legyen. Legfeljebb 20 karakter hosszú legyen. Csak betűt és számot tartalmazhat.
<i>Jelszó</i>	
	Legalább 8 karakter hosszú legyen. Legalább egy kis betűt tartalmazzon. Legalább egy nagy betűt tartalmazzon. Legalább egy számot tartalmazzon. Legalább egy speciális karaktert tartalmazzon.

2.1. táblázat. Regisztrációs követelmények

2.16. ábra. Regisztráció

2.8.3. Mini-játék gyűjtemény

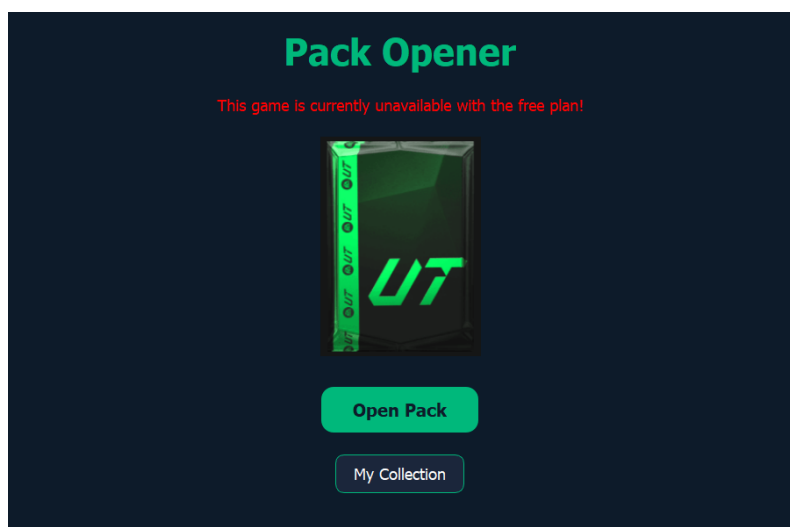
Sikeres bejelentkezést követően az alkalmazás a felhasználót a mini-játékok gyűjteményének felületére (2.17) navigálja, ahol az összes elérhető játék megtekinthető. A játékok kártyák formájában jelennek meg, melyeken az indexkép, a név, a rövid leírás, valamint egy gomb található a játék kiválasztásához. A bal felső sarokban elhelyezett kijelentkezés gomb segítségével a felhasználó visszatérhet az alkalmazás kezdőoldalára.



2.17. ábra. Mini-játékok gyűjtemény

2.8.4. FUT csomagnyitó

A mini-játékok gyűjteményének felületén (2.17 ábra) a „FUT csomagnyitó” játékkártyára kattintva az alkalmazás a játék kezdőoldalára navigálja a felhasználót. A játék jelenleg nem üzemel előfizetési problémák miatt, ezért csak a felület tekinthető meg. A „Csomagnyitás” gomb megnyomásakor a rendszer hibaüzenettel tér vissza. A navigáció továbbra is a bal felső sarokban található, ahol a felhasználó az előző oldalra léphet vissza.



2.18. ábra. Csomagnyitó játék

2.8.5. Kockázat játék

A mini-játékok gyűjteményének felületén (2.17 ábra) a „Kockázat játék” (angolul: Risk game) játékkártyára kattintva az alkalmazás a játék kezdőoldalára navigálja a felhasználót. A játék lényege, hogy a felhasználó támadásokat indíthat, és az egyes előrehaladásokat a tárolóba helyezheti. Amennyiben egy támadást sikeresen végrehajt, extra pontszámot kap. Minden további előrehaladással nő a leszerelés kockázata, amely esetén a felhasználó elveszíti az addig szerzett eredményeket.

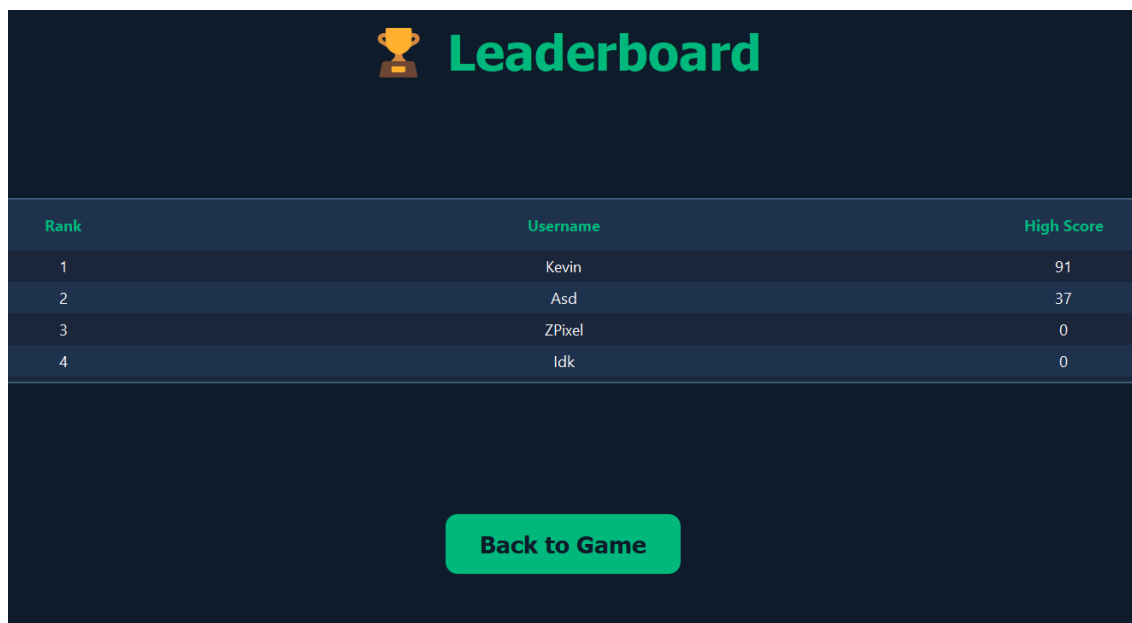
Az ablak elrendezése a következő:

- **Folyamatjelző sáv:** a felhasználó aktuális előrehaladásának vizuális megjelenítése.
- **Címkék:** szövegmezők adatok megjelenítésére
- **Gombsor:** a gombok funkcióit a (Táblázat 2.2) táblázat részletezi.
- **Szövegmező:** visszajelzést ad a játék során történő eseményekről.

Gomb	Leírás
<i>Tovább (Advance)</i>	Előrehaladás; a leszerelés kockázatát vállalva haladhatunk tovább, amely esetén minden eddigi eredmény elveszhet.
<i>Mentés (Hold)</i>	Az aktuális pontszám mentése a tárolóba.
<i>Új támadás (New Attack)</i>	Új támadás indítása.
<i>Törlés (Reset Progress)</i>	Minden adat törlése és a játék előrehaladásának visszaállítása.
<i>Ranglista (Leaderboard)</i>	Átnavigálás a ranglista felületre, ahol a legmagasabb eredmények tekinthetők meg.

2.2. táblázat. A Kockázat játék gombsorának leírása

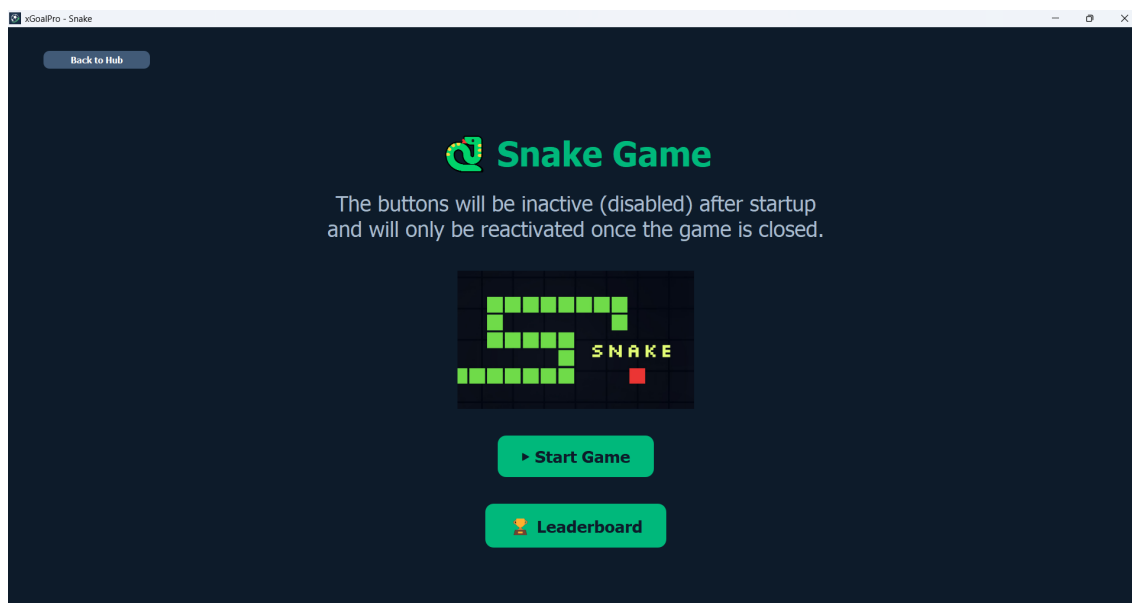
A mini-játékok gyűjteményéhez a felhasználó a bal felső sarokban található navigációs gomb segítségével térhet vissza. A játék rendelkezik ranglistával (2.19 ábra), ahol a legmagasabb eredmények tekinthetők meg.



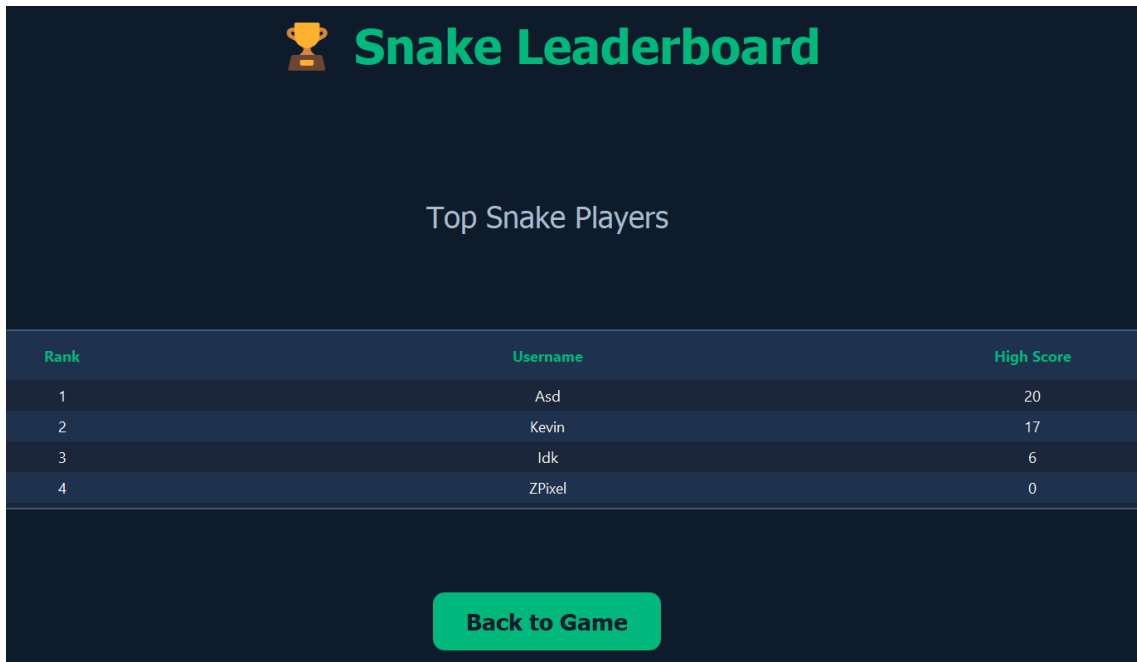
2.19. ábra. Kockázat játék ranglista

2.8.6. Kígyó játék

A mini-játékok gyűjteményének felületén (2.17 ábra) a „Kígyó játék” játékkártyára kattintva az alkalmazás a játék kezdőoldalára (2.20 ábra) navigálja a felhasználót. A felületen lehetőség van a játék elindítására, valamint a ranglista (2.21 ábra) megnyitására az erre szolgáló gombok segítségével. A mini-játékok gyűjteményéhez való visszatérés a bal felső sarokban elhelyezett navigációs gombbal történik.



2.20. ábra. Kígyó játék kezdőoldal

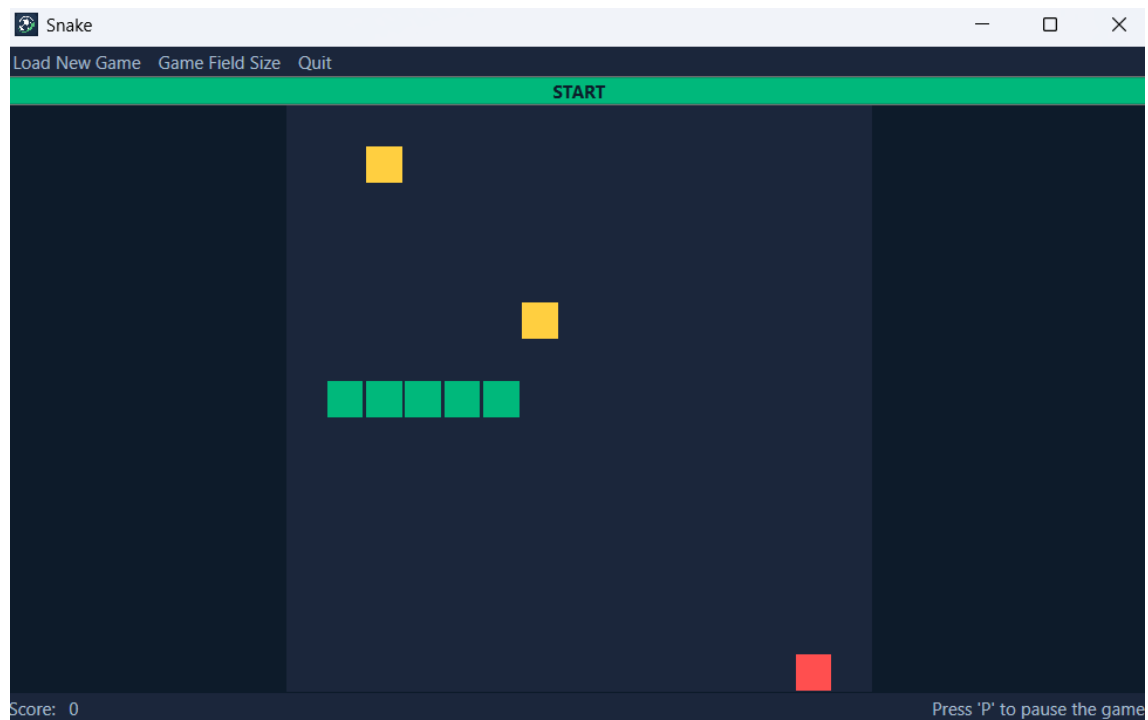


The screenshot shows a 'Snake Leaderboard' interface with a dark blue background. At the top, there is a trophy icon and the title 'Snake Leaderboard' in green. Below the title, the text 'Top Snake Players' is centered. A table lists the top four players with their rank, username, and high score. At the bottom, there is a green button labeled 'Back to Game'.

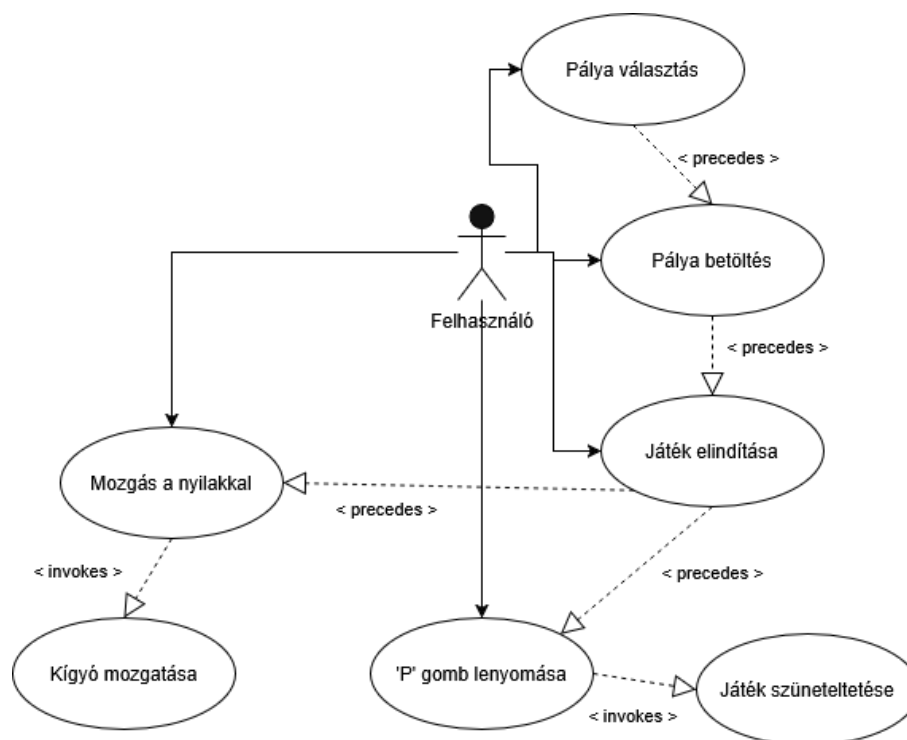
Rank	Username	High Score
1	Asd	20
2	Kevin	17
3	Idk	6
4	ZPixel	0

2.21. ábra. Kígyó játék ranglista

A játék egy klasszikus kígyó játék, amely falakkal és több különböző pályával egészül ki. A játék indítását követő nézetben (2.22 ábra) az irányítást az (2.23 ábra) ábra mutatja be.



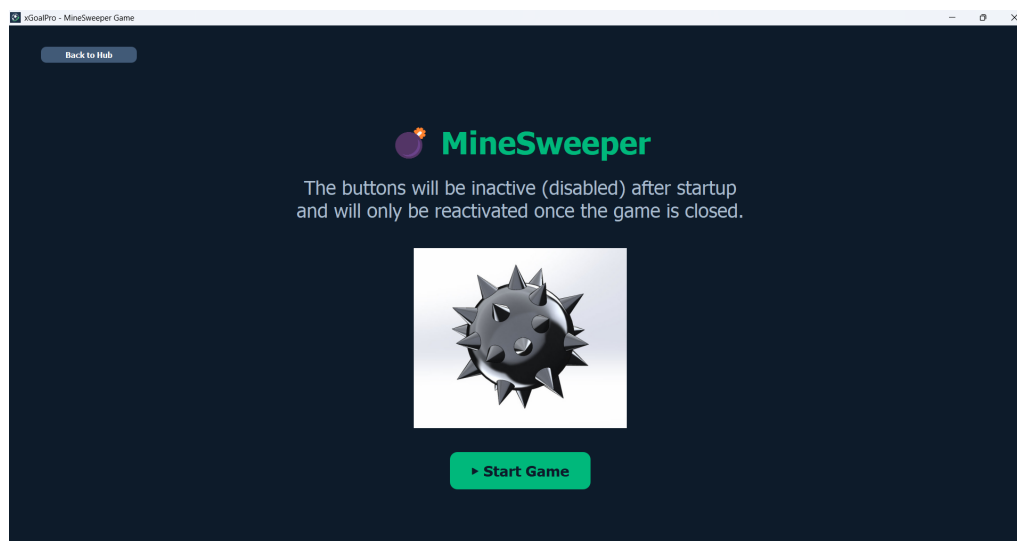
2.22. ábra. Kígyó játék felület



2.23. ábra. Kígyó játék használati eset diagram

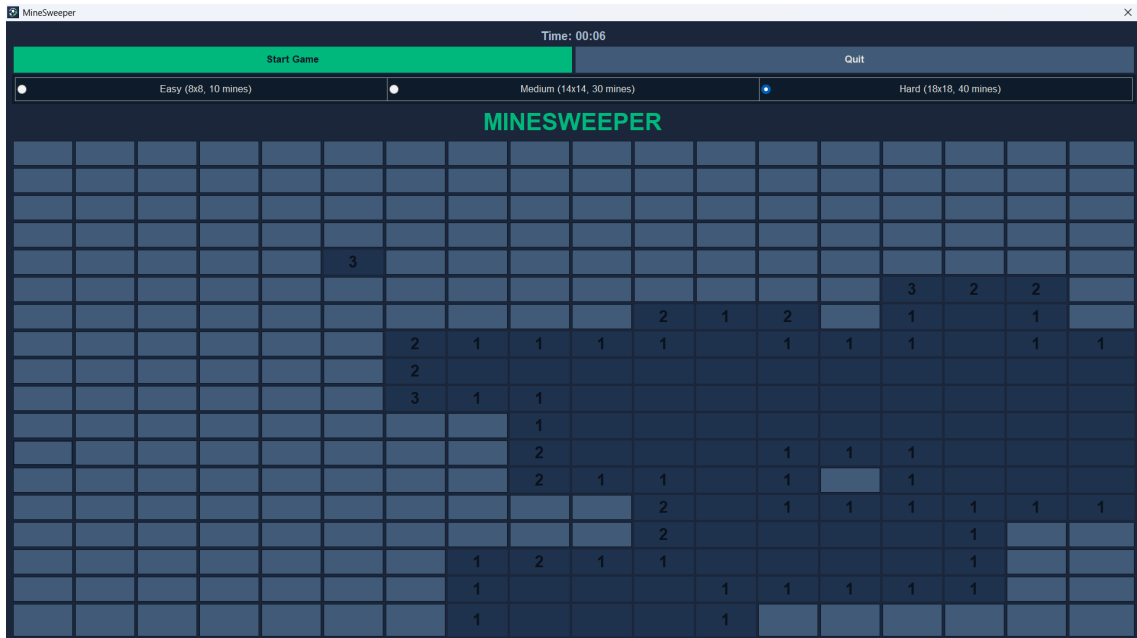
2.8.7. Aknakereső

A mini-játékok gyűjteményének felületén (2.17 ábra) az „Aknakereső” játékkártyára kattintva az alkalmazás a játék kezdőoldalára (2.24 ábra) navigálja a felhasználót. A felületen lehetőség nyílik a játék elindítására, illetve a bal felső sarokban található navigációs gombbal vissza lehet térni a mini-játékok gyűjteményének felületére.

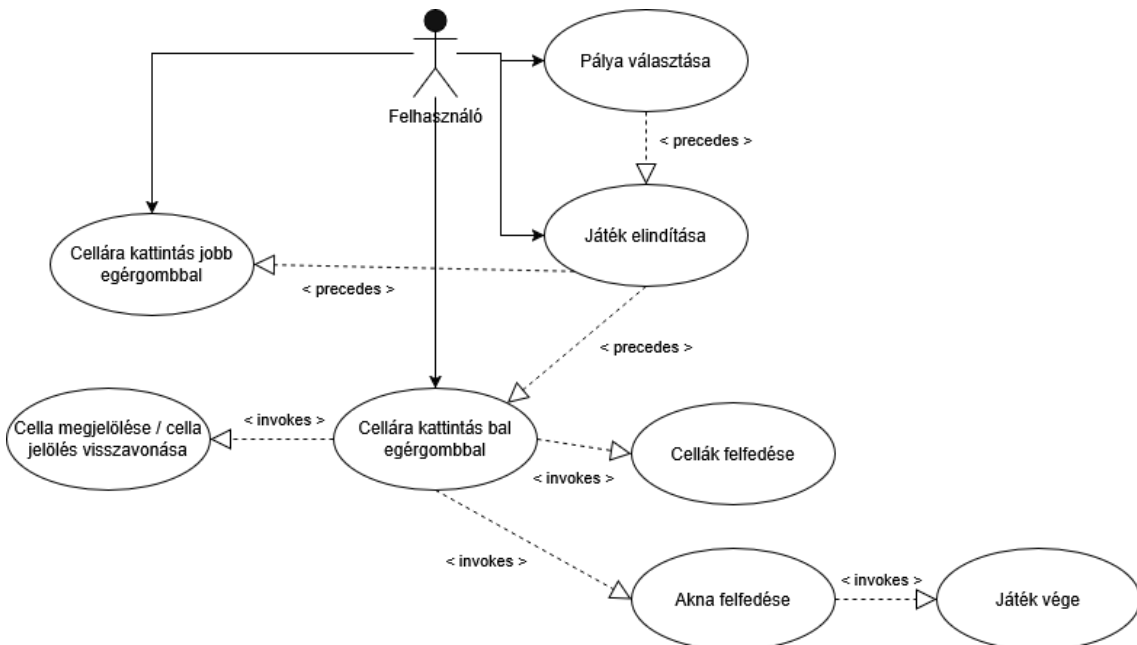


2.24. ábra. Aknakereső kezdőoldal

A játék indítását követően megjelenik az a felület (2.25 ábra), ahol maga a játékmenet zajlik. Az alkalmazás egy klasszikus aknakereső játékot valósít meg, amelyet több különböző pályaopció egészít ki. Az irányítás részletes bemutatását a (2.26 ábra) ábra szemlélteti.



2.25. ábra. Aknakereső felület

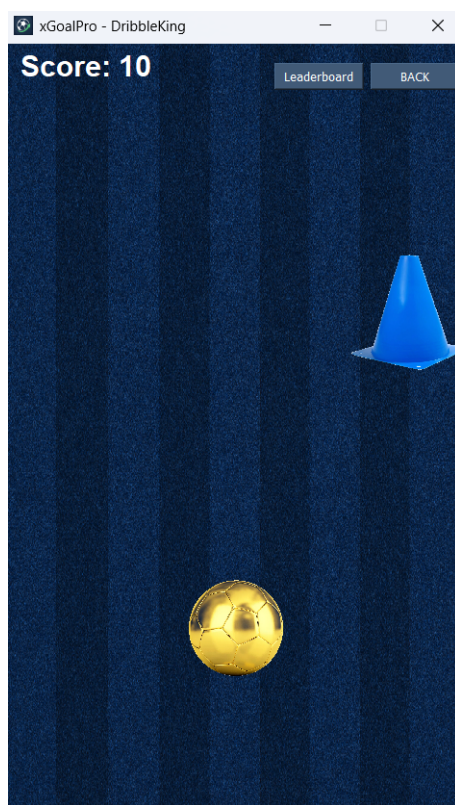


2.26. ábra. Aknakereső használati eset diagram

2.8.8. Cselezőkirály

A mini-játékok gyűjteményének felületén (2.17 ábra) a „Cselezőkirály” játékkártyára kattintva az alkalmazás a játék oldalára (2.27.a ábra) navigálja a felhasználót. A jobb felső sarokban találhatóak a navigációs gombok, amelyekkel vissza lehet térni a mini-játékok gyűjteményébe, illetve át lehet lépni a ranglista nézetbe (2.27.b ábra). A játékot az S billentyű lenyomásával lehet elindítani.

A cél, hogy elkerüljük a lehulló bójákat a véletlenszerűen pattogó labdával a három sáv valamelyikében. A sávok között az A és D billentyűkkel lehet mozogni.



(a) Cselezőkirály felület



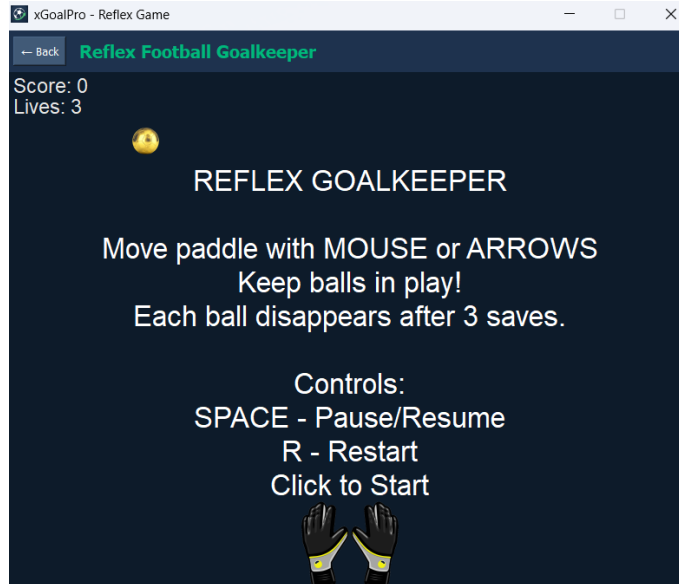
(b) Cselezőkirály ranglista

2.27. ábra. Cselezőkirály játék

2.8.9. Reflex játék

A mini-játékok gyűjteményének felületén a „Reflex játék” játékkártyára kattintva az alkalmazás a játék kezdőoldalára (2.28 ábra) navigálja a felhasználót. A játék lényege, hogy a felhasználó kesztyűvel elkapja a leeső labdákat. A játék bármilyen billentyű lenyomására vagy egérekattintásra elindul. A kesztyű mozgatása az egér vagy az A és D billentyűk segítségével történik, balra és jobbra. Amennyiben három

labdát nem sikerül elkapni, a játék véget ér. A játékot a szóköz (Space) billentyű megnyomásával lehet szüneteltetni. A visszavigáció a mini-játékok gyűjteményéhez a bal felső sarokban található navigációs gomb segítségével történik.



2.28. ábra. Reflex játék

2.9. Új modell tanítása

Az alkalmazás az előrejelzések elkészítéséhez előre betanított gépi tanulási modelleket használ, ugyanakkor lehetőséget biztosít új modellek tanítására is. Ez a funkció haladó felhasználók számára készült, és elsősorban azoknak ajánlott, akik rendelkeznek gépi tanulással kapcsolatos ismeretekkel. Ebből adódóan általános felhasználói kör számára nem javasolt a használata. A következőkben a leírás feltételezi, hogy a felhasználó megfelelő jártassággal rendelkezik a mesterséges intelligencia és a gépi tanulási módszerek területén.

A főoldalon (2.2 ábra) található „Tanítás” (angolul: Training) menüpontra kattintva megnyílik a tanítófelület (2.29 ábra), ahol jelenleg az SVR (Support Vector Regression) modell betanítása érhető el. A modell működéséről és elméleti háttéréről részletes ismertető található az Analytics Vidhya szakmai oldalán [3]. A főoldalra történő visszavigálás a felület bal felső sarkában elhelyezett vezérlőelemmel biztosított.

2.29. ábra. Paraméter szerkesztő ablak részlet

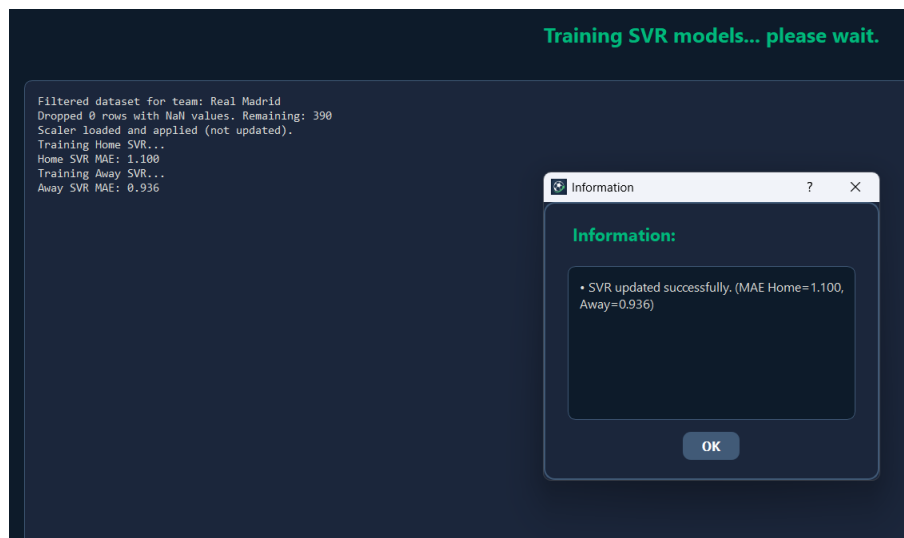
A tanítóablak beviteli mezőket tartalmaz, amelyek segítségével megadhatóak a modell paramétereinek intervallumai és konkrét értékei. A betanítási folyamat a felület alján elhelyezett indítógommbal kezdhető meg. A választható paraméterek részletes ismertetését a (Táblázat 2.3) táblázat tartalmazza. A tanítási folyamat megkezdése előtt célszerű áttekinteni az Analytics Vidhya szakmai oldalán elérhető összefoglalót [3], amely részletesen ismerteti az SVR modell működését és a paraméterek szerepét.

Paraméter	Leírás
<i>Csapat név</i>	Az adathalmaz csapatnév szerinti szűrésére szolgáló paraméter. A mező nem érzékeny az elírásokra, így kisebb pontatlanságok esetén is megtalálja a megfelelő nevet. Üresen hagyva nem alkalmaz szűrést.
<i>Dátum</i>	Az adathalmaz időbeli szűrését teszi lehetővé. A megadott dátum alapján csak a megfelelő időpontban lejátszott mérkőzések kerülnek kiválasztásra.
<i>C paraméter</i>	A modell regularizációs paramétere, amely meghatározza a hibák büntetésének mértékét. Nagy C érték esetén a modell szigorúan illeszkedik a tanítóadatokra, míg kisebb érték esetén nagyobb általánosítást tesz lehetővé.
<i>Gamma paraméter</i>	Az RBF magfüggvény érzékenységet szabályozó paraméter. Nagy gamma érték nagyon lokális illeszkedést eredményezhet, míg kis gamma érték simább, általánosabb regressziós függvényt hoz létre.

2.3. táblázat. Svr tanítás paraméterei

A tanítási folyamat során előfordulhat, hogy nagy mennyiségű adat feldolgozása válik szükségessé, ami jelentős terhelést róhat a rendszer memóriájára és processzorára. A szolgáltatás hatékony használatához ajánlott a Rendszerkövetelmények (szakasz 2.3) fejezetben leírt asztali számítógéphez hasonló, vagy annál nagyobb teljesítményű hardver alkalmazása. Fontos megjegyezni, hogy még erősebb konfiguráció esetén is több órát vehet igénybe egy modell teljes betanítása, amelyet a felhasználónak célszerű előre figyelembe vennie.

A szerkesztőfelületen (2.29 ábra) található indítógombra kattintva a rendszer egy új ablakra (2.30 ábra) navigál, ahol a felhasználó a tanítási folyamat teljes menétét végigkövetheti. A központi szövegmezőben jelennek meg a rendszerüzenetek — például a szűrések végrehajtásáról, a paraméterek beolvasásáról vagy a modell építésének egyes lépéseiről. A tanítási folyamat lezárultával a felület összegző üzenetet jelenít meg az eredményről, majd a szövegmező alatt elhelyezett gomb segítségével lehet visszatérni a szerkesztőablakhoz.

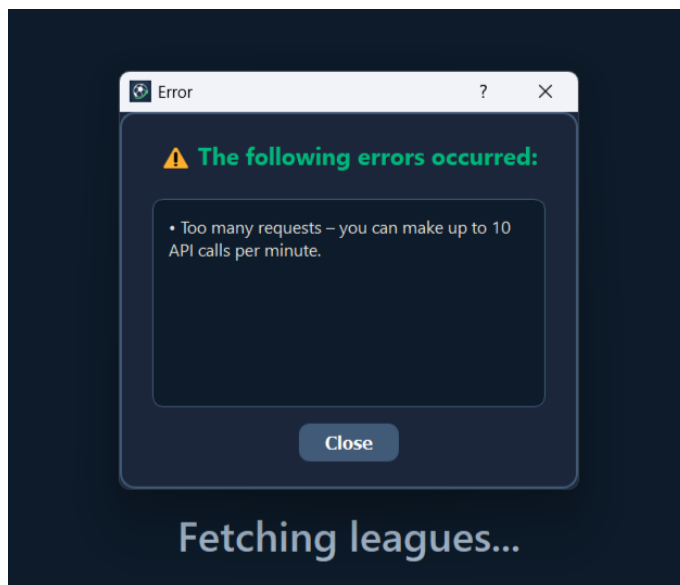


2.30. ábra. Tanító ablak részlet

2.10. Visszajelzések

Az alkalmazás üzenetdobozok (message boxok) segítségével jelenít meg minden olyan információt vagy hibaüzenetet, amely a felhasználói interakciók során releváns. Ezek a visszajelzések támogatják a felhasználót a rendszer működésének megértésében és a hibás műveletek korrekciójában. Például az előrejelző szolgáltatás esetében,

ha a felhasználó rövid időn belül túl sok kérést küld az alkalmazás felé, a rendszer egy hibaüzenetben (2.31. ábra) jelzi a túlterhelést.



2.31. ábra. Hiba ablak

3. fejezet

Fejlesztői dokumentáció

3.1. Követelmény elemzés

A szakdolgozat célja egy olyan felhasználóbarát alkalmazás fejlesztése, amely gépi tanulási és statisztikai modellek segítségével képes a futballmérkőzések várható végeredményének előrejelzésére. Az alkalmazásnak továbbá biztosítania kell a legfrissebb csapat- és játékosstatisztikák, valamint a korábbi predikciók és azok pontosságának visszakereshetőségét, ezáltal támogatva a felhasználók részletes elemzéseit és következtetéseik megalapozását. A felhasználói élmény növelése és a könnyed kapcsolódást a mini-játékok szolgáltatják.

Az alkalmazás elsődleges célcsoportját olyan futballrajongók és sportfogadók alkotják, akik döntéseiket statisztikai adatokra és modellezési eredményekre szeretnék alapozni. Emellett kiemelt szempont, hogy a felület a kevésbé tapasztalt felhasználók számára is könnyen érthető legyen; ennek érdekében az alkalmazás beépített súgókkal és magyarázatokkal segíti a sportágban járatlanabb felhasználók tájékozódását.

3.1.1. Funkcionális követelmények

Az alkalmazás fejlesztésének egyik központi eleme a funkcionális követelmények pontos meghatározása, mivel ezek határozzák meg a rendszer által nyújtott szolgáltatásokat és a felhasználók számára elérhető funkciók körét. Az alábbiakban a rendszer fő funkcionális komponensei kerülnek bemutatásra.

1. Adatkezelés

- A legfrissebb csapat- és játékosstatisztikák elérését több megbízható külső API biztosítja, amelyek lehetővé teszik, hogy az alkalmazás folyamatosan naprakész adatokkal működjön.
- Az előrejelzésekhez kapcsolódó adatok, valamint a felhasználók által generált információk hosszú távú tárolását egy relációs adatbázis biztosítja. Ez a megoldás lehetővé teszi az adatok konzisztens és strukturált kezelését.

2. Előrejelzések

- A predikciók elkészítéséhez az alkalmazás többféle gépi tanulási modellt használ. A felhasználók tetszés szerint választhatnak a különböző algoritmusok közül, illetve igény esetén egyszerre több modellt is kiválaszthatnak az előrejelzéshez.
- A különböző modellekkel készült előrejelzések eredményei az adatbázisban kerülnek eltárolásra, ami lehetővé teszi azok későbbi összehasonlítását, teljesítményelemzését és a prediktív pontosság hosszú távú nyomon követését.
- A predikciós modellek nem konkrét, determinisztikus kimeneteket adnak vissza, hanem valószínűségi becsléseket. A valószínűségi alapú előrejelzések lehetővé teszik, hogy a felhasználó megalapozottabb, a valósághoz közelítő eredményekre támaszkodhasson.

3. Megjelenítés

- A legfrissebb csapat- és játékosadatok felhasználóbarát, áttekinthető és vizuálisan vonzó módon kerülnek megjelenítésre az alkalmazás felületén, elősegítve a gyors tájékozódást.
- Az előrejelzések eredményeiből grafikonok készülnek, amelyek vizuális támogatást nyújtanak az elemzésekhez. A grafikonok részletes címkézéssel és magyarázó szövegekkel történő kiegészítése biztosítja az adatok könnyebb értelmezhetőségét.
- A közelgő mérkőzések időrendben, naptárstruktúrában kerülnek megjelenítésre, lehetővé téve a felhasználók számára az események egyszerű és áttekinthető nyomon követését.

4. Mini-játék funkciók

- A rendszer lehetővé teszi új fiók létrehozását a felhasználói adatok tárolására, valamint a meglévő fiókba történő belépést.
- A felületen egy mini-játék választó menü található, amely több, általánosságban labdarúgás témájú játék közül teszi lehetővé a választást.
- Egyes mini-játékoknál a pontszámok az adatbázisba kerülnek mentésre a későbbi ranglistás megjelenítés céljából.

3.1.2. Nem funkcionális követelmények

Az alapvető funkciókon túl a nem funkcionális követelmények határozzák meg a rendszer teljesítményének szintjét. A teljes felhasználói élmény biztosítása érdekében a fejlesztés során elengedhetetlen a megfelelő követelmények pontos meghatározása. A következőkben a nem funkcionális követelményeket részletezem.

Használhatóság: Az alkalmazás felületének átláthatónak és könnyen kezelhetőnek kell lennie minden felhasználó számára, beleértve a labdarúgásban kevésbé jártasakat is.

Teljesítmény: Az adatok betöltése, az ablakok közötti váltás és az előrejelzések számítása legfeljebb 5 másodpercet vegyen igénybe.

Biztonság: A felhasználói adatok jelszóval védettek legyenek, a jelszavak hash-elt formában, a mai szabványoknak megfelelően tárolódjanak.

Karbantarthatóság: Az alkalmazás moduláris felépítésű, több rétegű architektúrával, a könnyű karbantartás érdekében. Továbbá az egyes függvények és osztályok a tiszta kód elvei szerint dokumentáltak legyenek.

Bővíthetőség: Az alkalmazás skálázható, így új ligák, adatbázisok vagy mini-játékok hozzáadása egyszerűen megvalósítható.

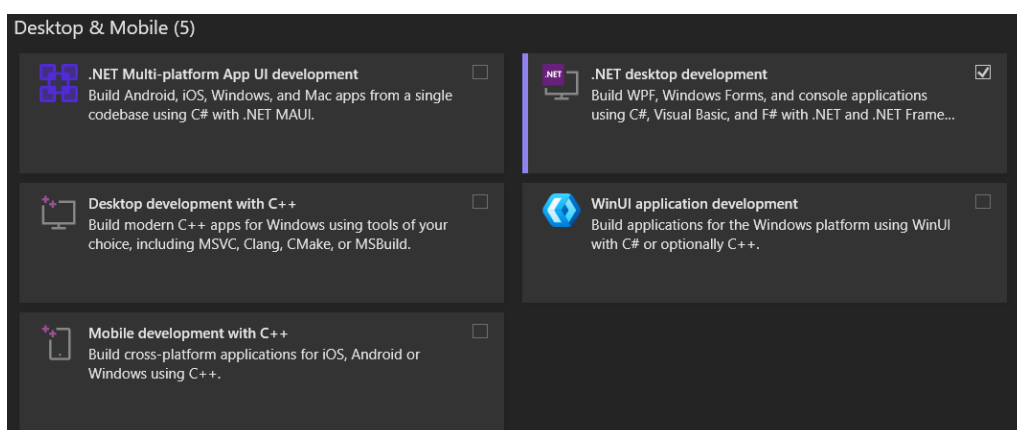
Megbízhatóság: Az esetleges hibák megfelelően kezelve vannak, és a felhasználók számára világos, érthető módon kerülnek megjelenítésre.

3.2. Fejlesztői környezet és technológiák

Az alkalmazás fejlesztéséhez és a modellek tanításához Windows 11 Home operációs rendszerű személyi számítógépet használtam. A kód szinte teljes egé-

szében Pythonban¹ íródott. A kliensoldali kód a Qt keretrendszer Python API-jában [4], a PyQt5-ben készült, míg a serveroldali kód két játék kivételével szintén Pythonban van megvalósítva. A két kivétel a kígyó játék, amely Windows Presentation Foundation (WPF)², és az aknakereső, amely Windows Forms (WF)³ környezetben készült. A modellek tanítása a Pythonhoz elérhető kiegészítő modulok segítségével történt.

A Python-kód fejlesztése a PyCharm Community és a Visual Studio Code környezetekben zajlott, míg a WPF és WF kódok a Microsoft Visual Studio 2022 Community környezetben készültek. A Python fejlesztéshez legalább a 3.11.1-es Python interpreter szükséges, valamint a kiegészítő csomagok telepítése (Táblázat 3.1), amely a pip csomagkezelő segítségével egyszerűen elvégezhető (3.1 kód). A Visual Studio használatához a .NET Desktop Development komponens telepítése szükséges; hiány esetén újratelepítés útján kell pótolni (3.1 ábra).



3.1. ábra. Visual Studio komponens telepítés

```
1 pip install PyQt5 qasync aiohttp joblib python-dateutil numpy scipy
   pandas Pillow understat asyncpg aiosqlite pytest matplotlib
   scikit-learn xgboost optuna soccerdata
```

3.1. forráskód. A szükséges Python csomagok telepítése

Az adatbázis-kezeléshez SQLite került alkalmazásra, de a későbbi kihelyezés lehetősége miatt PostgreSQL felhőalapú adatbázis is rendelkezésre áll. A gépi tanulások modellekről, API-król és az adatállományokról, valamint a perzisztencia és mester-séges intelligencia megvalósításáról a későbbi fejezetekben lesz részletesen szó.

¹<https://www.python.org/>

²<https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/>

³<https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/>

Az alkalmazás a `main.py` fájl futtatásával indítható. A WPF és WF küdök a `.sln` fájlra kattintva, a Visual Studio környezetben belül indíthatók.

Csomag	Leírás
<i>PyQt5</i>	GUI fejlesztéshez: ablakok, gombok, táblázatok, jelzők (signals)
<i>qasync</i>	Async/await integráció Qt eseményhurokkal (PyQt/PySide)
<i>aiohttp</i>	Aszinkron HTTP kliens és szerver, API hívásokhoz
<i>joblib</i>	Nagy objektumok (pl. modellek) mentése és betöltése, párhuzamos számítások
<i>python-dateutil</i>	Dátumok kezelése
<i>numpy</i>	Többdimenziós tömbök kezelése
<i>scipy</i>	Valószínűségi számítás és statisztika Python csomag
<i>pandas</i>	Strukturált adatok kezelése
<i>Pillow</i>	Képek kezelése
<i>understat</i>	Understat statisztikák lekérésére API
<i>asynpg</i>	PostgreSQL aszinkron kliens
<i>aiosqlite</i>	SQLite aszinkron kliens
<i>pytest</i>	Tesztelés, egységteszt futtatása
<i>matplotlib</i>	Grafikonok készítése
<i>scikit-learn</i>	Gépi tanulás: modellezés, skálázás, SVM, regressziók, keresések
<i>xgboost</i>	Gradiens boosting modellek
<i>optuna</i>	Paraméter optimalizáló
<i>soccerdata</i>	Labdarúgás adatokra és Elo pontra API

3.1. táblázat. Python csomagok leírása

A Python nyelv választását elsősorban a gépi tanulási modellek egyszerű tanítása indokolta, amelyet a nyelvhez elérhető kiegészítő modulok tesznek lehetővé. Emellett a Python mellett szólt az is, hogy az adatbázis- és felhasználói felület-technológiák könnyen integrálhatók a nyelvbe. Az SQLite egyszerűen illeszthető Python környezetbe, míg a Qt Python API-ja (PyQt5) hatékony eszközt biztosít a felhasználói felület kialakításához.

3.3. A rendszer architektúrális felépítése

Az alkalmazás az objektumorientált programozás paradigmája szerint készült, különös hangsúlyt fektetve a SOLID elvek betartására. A fejlesztés során kiemelt szerepet kapott az egyetlen felelősség elve, amely a funkcionális szétválasztást biztosítja. Ennek eredményeként a szoftver könnyebben karbantartható, az egyes komponen-

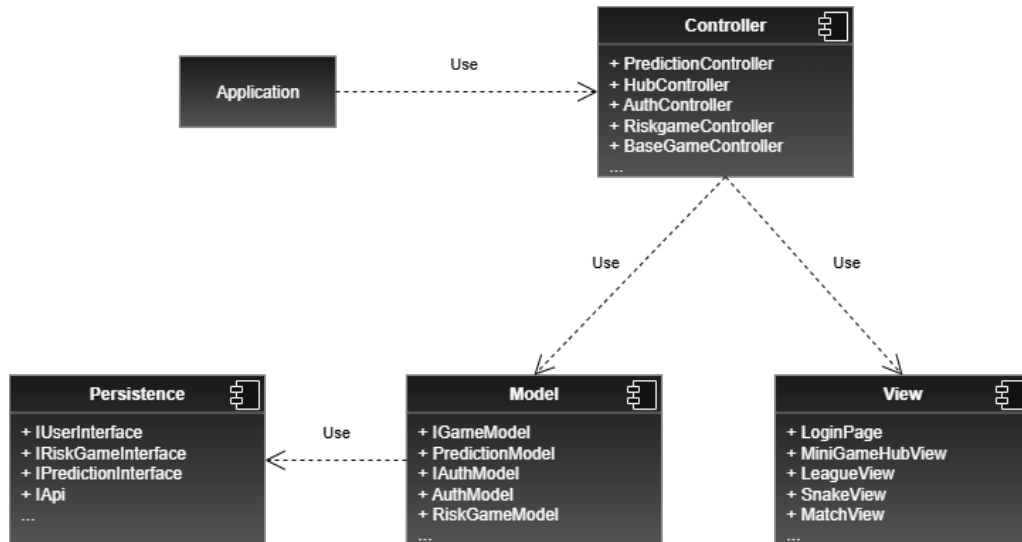
sek pedig egyszerűen lecserélhetők vagy továbbfejleszthetők akár más technológiák alkalmazásával is.

A nyílt/zárt elv szintén meghatározó az alkalmazás struktúrájában, különösképpen a bővíthetőség szempontjából. A rendszer úgy került kialakításra, hogy a meglévő kód módosítása nélkül lehessen új funkciókat — például további bajnokságokat — hozzáadni, ezáltal támogatva a hosszú távú fejleszthetőséget és skálázhatóságot.

1. Definíció. Az Egyetlen Felelősség Elve (Single Responsibility Principle) kimondja, hogy egy programegységnek (például komponensnek, osztálynak vagy metódusnak) csak egyetlen felelőssége lehet, vagyis csak egy jól körülhatárolható tárgykörért felelhet. A felelősség olyan logikailag összetartozó műveletek és adatok együttese, amely egy adott üzleti szerepkör követelményeinek teljesítését szolgálja, és amely egy önállóan változtatható tényezőként értelmezhető („egy ok a változásra”). [5]

2. Definíció. A Nyílt/Zárt Elv (Open/Closed Principle) kimondja, hogy a programegységeknek nyitottnak kell lenniük a kiterjesztésre, ugyanakkor zártnak a módosításra. Ennek megfelelően a program szerkezetének úgy kell kezelnie az új funkcionális hozzáadását, hogy azok megvalósítása ne igényelje a meglévő kód módosítását, hanem elegendő legyen új komponensek vagy modulok létrehozása. Ezt az elvet általában absztrakt őssz osztályok, interfészek, illetve polimorfizmus alkalmazásával lehet hatékonyan érvényesíteni. [6]

A szerepkörök szétválasztása és a könnyebb fejlesztés érdekében az alkalmazás háromrétegű Modell–Nézet–Vezérlő (MVC) architektúrában készült (3.2 ábra), amelyet egy külön perzisztenciaréteg egészít ki. Ez a megközelítés jól elkülöníti az adatok kezelését, az üzleti logikát, a felhasználói felületet és a vezérlést, valamint biztosítja, hogy az egyes komponensek könnyen lecserélhetők legyenek más megvalósításokra — például a perzisztenciaréteg esetében létezik SQLite- és PostgreSQL-alapú implementáció is.



3.2. ábra. Az alkalmazás csomagdiagramja

Modell réteg: Ez a réteg felelős az adatok reprezentálásáért és kezeléséért. Ide tartoznak a csapatokat, játékosokat, mérkőzéseket és előrejelzéseket kezelő osztályok. Ezen kívül itt találhatóak a gépi tanulási modellek, amelyek a predikciókat generálják, valamint az összes minijáték üzleti logikája is itt valósul meg.

Vezérlő réteg: A vezérlő a nézet, a modell és az alkalmazás közötti mediátorszerepet tölti be. Feladata a kommunikáció és az adatáramlás biztosítása, valamint a folyamatok megfelelő levezénylése.

Nézet réteg: A nézet a vezérlőtől kapott adatok alapján végzi a grafikus megjelenítést. Feladata, hogy a PyQt5 elemekre építve felhasználóbarát és esztétikus grafikus felületet biztosítson az interakciókhoz.

Perzisztencia réteg: Ez a réteg felel az adatok hosszú távú tárolásáért és visszakereséséért. Biztosítja a kapcsolatot a modell és az adatbázis, valamint a modell és az API-k között.

3.4. Az alkalmazás belépési pontja

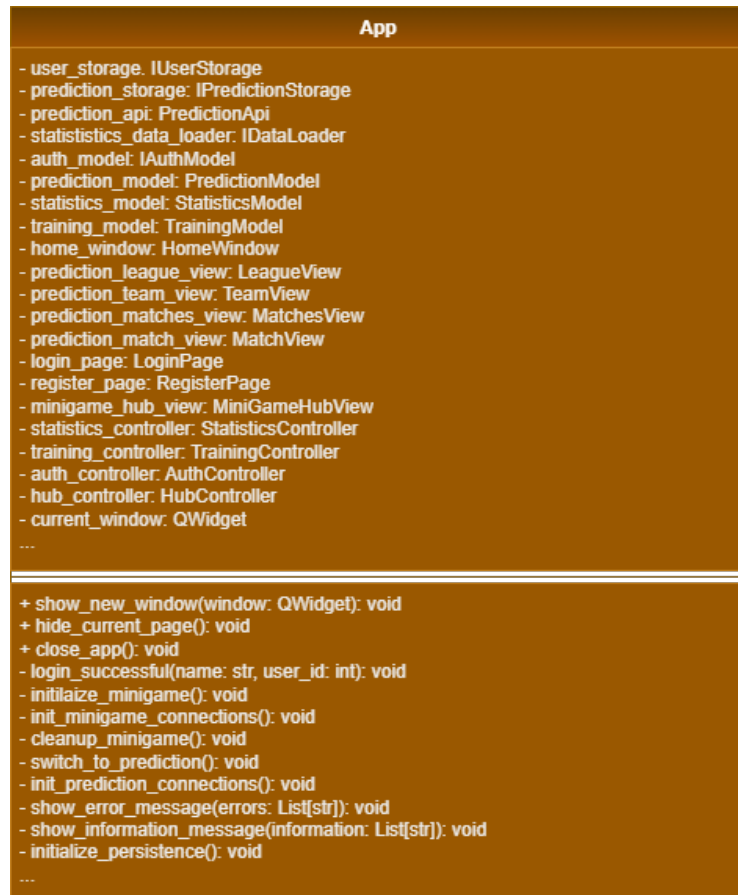
A `main.py` az alkalmazás belépési pontja, ennek futtatásával indul a szoftver. A PyQt rendszerek esetén az első lépés egy `QApplication` példány létrehozása, amely kezeli az eseményhurokot. Általában a `QApplication _exec_()` módszerával indítható az alkalmazás, azonban a szoftver több külső API-val dolgozik, amelyek aszinkron

működést igényelnek, például a töltőképernyőhöz, amíg az adatok lekérésre kerülnek. Az aszinkron műveleteknek saját, a PyQt-tól független eseményhurokja van, melyre a qasync csomag nyújt megoldást: a QEventLoop segítségével az aszinkron műveletek és a PyQt eseményhurokja összehangolható, így az alkalmazás problémamentesen futtatható a `run_forever()` metódussal (3.2 kód).

```
1 if __name__ == "__main__":
2     try:
3         app = QApplication(sys.argv)
4         loop = QEventLoop(app)
5         asyncio.set_event_loop(loop)
6
7         # Initialize the application asynchronously
8         main_app = App()
9
10        app.aboutToQuit.connect(loop.stop)
11
12        with loop:
13            try:
14                loop.run_forever()
15            finally:
16                loop.run_until_complete(main_app.close_app())
17                loop.close()
```

3.2. forráskód. A main.py, az alkalmazás belépési pontja

A kódban létrejön egy példány az App osztályból (3.3 ábra), amely az alkalmazás fő vezérlő eleme. Az osztály feladata a kezdeti rétegek inicializálása, a jelzések közötti kapcsolatok felépítése, valamint a tárhelyek kezelésének elindítása. A program futása során az App osztály fő feladata az ablakok közötti váltás és az üzenetek megjelenítése. A további részfeladatokat az alvezérlők biztosítják.



3.3. ábra. App osztálydiagramja

Az App osztály főbb metódusai a következők:

- `__init__`: az összes réteg inicializálása
- `show_new_window(window)`: új ablak megjelenítése
- `_login_successful`: sikeres bejelentkezés után a mini-játékok inicializálása és váltás a mini-game felületre
- `_init_minigames_connections`: a mini-játékokhoz használt jelzők összekapcsolása a megfelelő kezelővel
- `_cleanup_minigame`: a mini-játék által használt tárhely felszabadítása
- `_switch_to_prediction`: váltás a predikciós ablakra
- `_initialize_prediction`: a predikció kezdeti inicializálása
- `__show_error_message`: hibaüzenet megjelenítése
- `__show_information_message`: információs üzenet megjelenítése
- `__initialize_persistence`: a perzisztencia inicializálása

A program futásának végén kiemelt fontosságú a meglévő kapcsolatok és szá-

lak bezárása. Ennek érdekében a program bezárásakor az App osztálypéldány `close_app()` metódusa bontja a meglévő kapcsolatokat, végighaladva minden alvezlőn is.

3.5. Perzisztencia

A felhasználói adatok és az előrejelzések eredményei adatbázisban kerülnek tárolásra a későbbi feldolgozás érdekében. A két külön szolgáltatáshoz két külön SQLite adatbázis került kialakításra. A model réteg az adatbázissal kommunikáló osztályokat a `StorageInterface.py` segítségével éri el, így az absztrakció biztosított, és lehetővé válik különböző adatbázis-megoldások, például PostgreSQL, egyszerű hozzáadása. Ezen felül az alkalmazás további adatokat kettő különböző API segítségével kér le.

3.5.1. Predikció adatbázis

A predikciós adatbázis két táblát tartalmaz: az egyik az előrejelzések, a másik az Elo pontok rögzítésére szolgál. A program indításakor az adatbázissal a kapcsolat automatikusan létrejön.

1. A predictions tábla mezői:

- `id`: a sor egyedi azonosítója
- `match_id`: a mérkőzés egyedi azonosítója
- `model`: az előrejelzésekhez használt modellek azonosító kódokkal rendelkeznek; például ha az első két modellt használja az alkalmazás, a kód 1100 jelöléssel kerül rögzítésre.
- `home_id`: a hazai csapat egyedi azonosítója
- `home_name`: a hazai csapat neve
- `home_xg`: a hazai csapat várható góljára adott predikció
- `away_id`: az idegenbeli csapat egyedi azonosítója
- `away_name`: az idegenbeli csapat neve
- `away_xg`: az idegenbeli csapat várható góljára adott predikció
- `home_p`: a hazai győzelemre adott predikció százalékban
- `draw_p`: a döntetlenre adott predikció százalékban
- `away_p`: az idegenbeli győzelemre adott predikció
- `real_home_score`: a hazai gólok száma (mérkőzés végeredménye)

- `real_away_score`: az idegenbeli gólok száma (mérkőzés végeredménye)

2. Az `eloScores` pont tábla mezői:

- `name`: a csapat hosszú neve
- `eloName`: a csapat rövid neve
- `elo`: az elo mező a csapat erősségét számszerűsítő pontszámot tartalmazza

A `home_xg`, `away_xg`, `home_p`, `draw_p` és `away_p` mezők a gépi tanulási modellek által visszaadott értékek, amelyek mind valószínűségi becsléseket tartalmaznak, ezért lebegőpontos típusúak. A táblákkal kapcsolatos műveleteket az `SQLitePredictionStorage.py` modul valósítja meg (3.5 ábra).

3.5.2. Mini-játék adatbázis

A mini-játék adatbázis öt táblát tartalmaz: az egyik a bejelentkezési adatokat tárolja, míg a maradék négy a különböző játékokhoz kapcsolódó adatokat. A program indításakor az adatbázissal a kapcsolat automatikusan létrejön.

1. A `users` tábla mezői:

- `id`: a sor egyedi azonosítója
- `username`: a felhasználó neve
- `email`: a felhasználó email címe
- `password_salt`: a mező a jelszó hash-elése után hozzáadott további adatdarabot tárolja, amely növeli a jelszó biztonságát.
- `password_hash`: a mező a felhasználó jelszavának hash-elte változatát tárolja.
- `created_at`: a létrehozás dátuma
- `updated_at`: a módosítás dátuma

2. A `snakeScores` tábla mezői:

- `id`: a sor egyedi azonosítója
- `user_id`: a felhasználó egyedi azonosítója
- `score`: a jelenlegi elért pontszám
- `high_score`: a legnagyobb elért pontszám

3. A `riskScore` tábla mezői:

- id: a sor egyedi azonosítója
- user_id: a felhasználó egyedi azonosítója
- bank: a jelenlegi elért pontszám
- high_score: a legnagyobb elért pontszám

4. A dribbleScore tábla mezői:

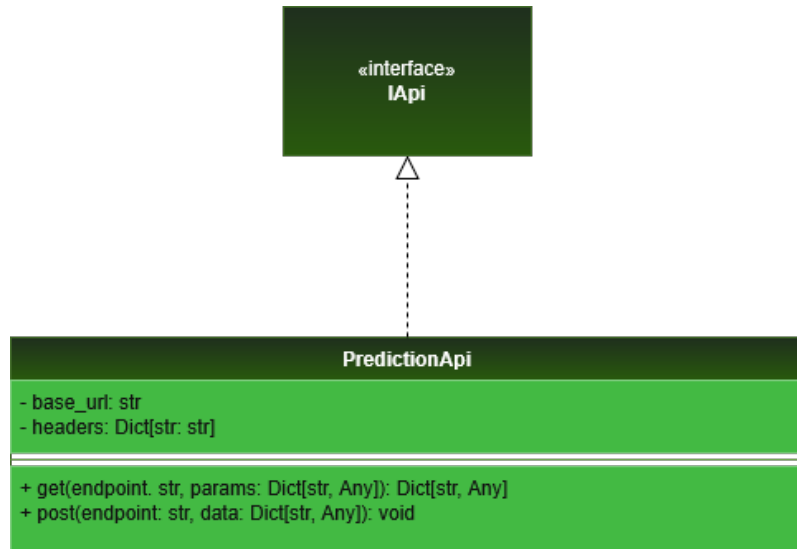
- id: a sor egyedi azonosítója
- user_id: a felhasználó egyedi azonosítója
- score: a jelenlegi elért pontszám
- high_score: a legnagyobb elért pontszám

5. A collections tábla mezői:

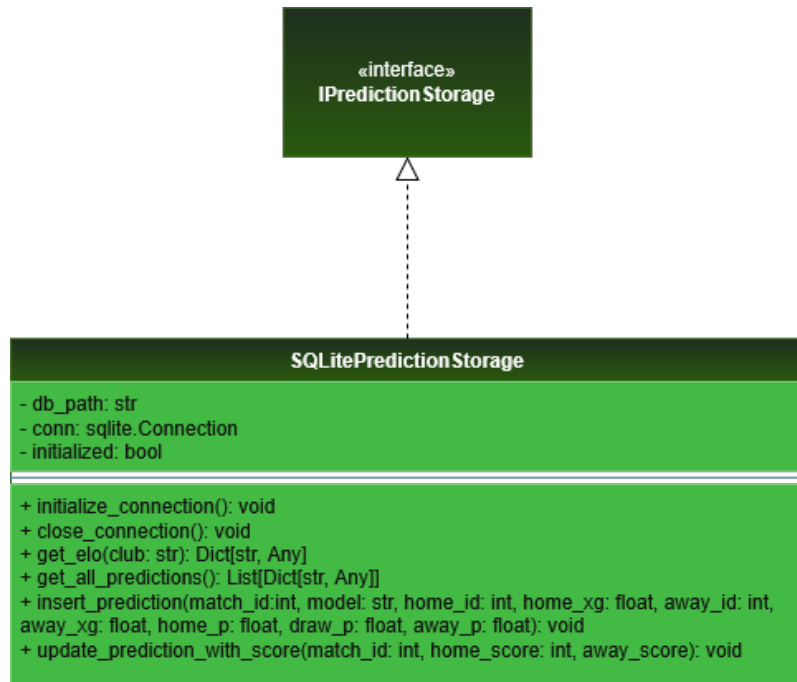
- id: a sor egyedi azonosítója
- user_id: a felhasználó egyedi azonosítója
- card_id: a kártya egyedi azonosítója
- acquired_at: a létrehozás dátuma

A user tábla adatainak olvasását és módosítását a SQLiteUserStorage.py modul valósítja meg. A gyűjteményekhez és a nyitó játékhöz tartozó adatokat a SQLiteCollectionStorage.py kezeli. A fennmaradó mini-játékok — a Risk Game, a Snake Game és a Dribble King — tábláinak műveleteit a SQLiteRiskGameStorage.py, SQLiteSnakeGameStorage.py és SQLiteDribbleKingStorage.py modulok látják el.

Ezek a modulok felelnek a megfelelő táblák teljes körű adatkezeléséért, így többek között az adatok beszúrásáért, frissítéséért, törléséért és lekérdezéséért, miközben biztosítják az egységes és jól elkülönített adattárolási réteget az alkalmazás számára.



3.4. ábra. Részleges perzisztencia osztálydiagram: api hívások



3.5. ábra. Részleges perzisztencia osztálydiagram: előrejelzés adatbázis

3.5.3. Külső API-k

A predikció és statisztika menükben megjelenő bajnokság-, csapat- és mérkőzés-adatokat az alkalmazás a football-data.org [7] API-ból szerzi be. Ugyanez az API biztosítja az előrejelzésekhez szükséges statisztikai információkat is, például az elmúlt öt mérkőzés formáját. Az API végpontjaihoz tartozó részletes dokumentáció és a visszatérési struktúra az API weboldalán érhető el. A hívásokat a PredictionApi

osztály (3.4 ábra) valósítja meg, amelynek a `get` metódusa paraméterként kapja a kívánt végpontot és az ahhoz tartozó paramétereket.

Az alkalmazás egy másik API-t is használ, amely az aktuális Elo-pontszámok lekéréséért felel. Ezt a funkciót a `soccerdata` csomag [8] valósítja meg, amely a csapatok erősségi mutatóit szolgáltatja.

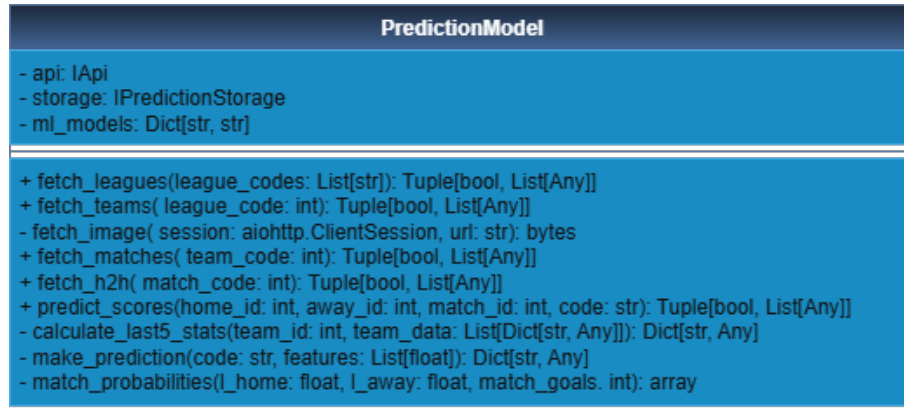
A rendszer működését két külön frissítő script támogatja: az egyik az Elo-pontszámok, a másik pedig a valós mérkőzések eredmények aktualizálását végzi. Ezek a scriptek frissítik az adatbázis megfelelő tábláit, biztosítva, hogy az alkalmazás minden modulja naprakész információkkal dolgozzon. Jelenleg ezt a frissítési folyamatot manuálisan kell elvégezni. Egy lehetséges továbbfejlesztési irány, hogy a rendszer szerveroldali környezetben automatikusan, időzítetten frissítse az adatokat — például `cron job` vagy más ütemezett feladatkezelő használatával.

3.6. A modell réteg

A perzisztens tárolóból lekért adatokat a modell réteg dolgozza fel. Ebben a rétegben történik az adatok értelmezése, átalakítása és előkészítése, valamint itt valósul meg az alkalmazás üzleti logikája, biztosítva az adatok megfelelő kezelhetőségét a további feldolgozás és a felhasználói interakciók számára.

3.6.1. Predikciós modell

A `PredictionModel` osztály (3.6 ábra) feladata az, hogy az előrejelzések helyes végrehajtásához az adatokat előkészítse a felsőbb rétegek számára. Egy példány létrehozásakor meg kell adni a kiválasztott bajnokságok kódjait, amelyekből a modell dolgozni fog, továbbá az API-t és a tárolási helyet, ahonnan az információk érkeznek és ahova a feldolgozott adatok mentésre kerülnek. A létrehozáskor meg kell adni a betanított gépi tanulási modellek elérési útvonalát is.



3.6. ábra. A predikciós modell osztálydiagramja

A legtöbb metódus az API-ból visszakapott adatokat rendszerezi és továbbítja a felsőbb rétegeknek feldolgozásra. A `predict_scores` metódus végzi az alkalmazás kulcsfeladatát, azaz a mérkőzések előrejelzését. A metódus a példány létrehozásakor megadott modelleknek adja át paraméterként az API-ból származó mérkőzésadatokat, és visszaadja a várható gólok becslését. A hazai és idegenbeli győzelem valószínűségének meghatározásához a szoftver Poisson-regressziót alkalmaz, feltételezve, hogy a várható gólok Poisson-eloszlást követnek.

3. Definíció (Poisson-eloszlás). A **Poisson-eloszlás** [9] egy diszkrét valószínűségi eloszlás, amely azt írja le, hogy egy adott idő- vagy területintervallumban hány esemény történik, ha az események átlagos gyakorisága ismert és függetlenek egymástól.

Ha X Poisson-eloszlású változó, akkor az $X = k$ valószínűsége:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

ahol:

- $\lambda > 0$ az események **átlagos száma** az adott intervallumban,
- k a **lehetséges eseményszám** (nemnegatív egész szám).

A Poisson-eloszlás várható értéke és szórása:

$$\mathbb{E}[X] = \lambda, \quad \sigma_X = \sqrt{\lambda}.$$

A Poisson-eloszlás ritkán előforduló események, például tornádók éves gyakorisága vagy, a jelen esetben, 90 perc alatt szerzett gólok számának modellezésére

alkalmas. A `__match_probabilities` (3.3 kód) metódus számítja ki a mérkőzés kimeneteleinek valószínűségeit. A metódus egy mátrixot hoz létre, ahol egy cella értéke az adott eredmény valószínűségét tartalmazza. A `lambda` paraméter a betanított modellekből származó becsült várható gól, míg `k` az index, amely az adott mérkőzés eredményét jelöli. [10]

Például, ha `k=(2,1)` és `lambda=(1.5,0.9)`, akkor az eloszlás azt számítja ki, hogy mekkora a valószínűsége annak, hogy a hazai csapat 1.5 várható gól mellett 2 gólt szerez, míg az idegenbeli csapat 0.9 várható gól mellett 1 gólt szerez.

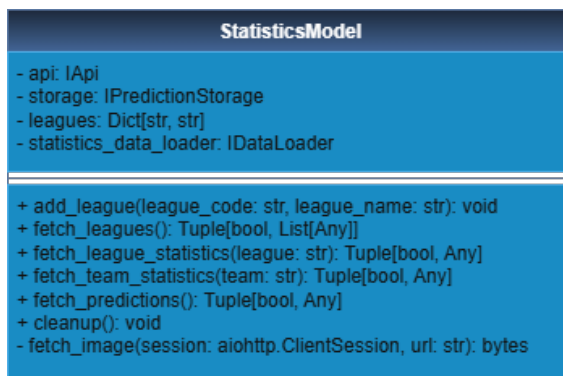
Külön számítás történik a hazai és az idegenbeli gólokra, amelyeket a végén összeszorozunk, mivel a gólok általában függetlenül esnek a két csapat között. A mátrix átlója a döntetlen valószínűségeket tartalmazza, míg az átló alatti és feletti háromszög a hazai, illetve az idegenbeli győzelem valószínűségeit.

```
1 def __match_probabilities(l_home: float, l_away: float, max_goals:
   int = 10) -> Optional[np.array]:
2     """Calculate match score probabilities using a Poisson
       distribution.
3
4     Args:
5         l_home (float): Expected goals for home team.
6         l_away (float): Expected goals for away team.
7         max_goals (int): Maximum goals considered for
           probability matrix.
8
9     Returns:
10        np.array: Matrix of probabilities for all possible
            score combinations.
11    """
12    probs = np.zeros((max_goals + 1, max_goals + 1))
13    for i in range(max_goals + 1):
14        for j in range(max_goals + 1):
15            probs[i, j] = poisson.pmf(i, l_home) * poisson.pmf(
                j, l_away)
16    return probs
```

3.3. forráskód. A valószínűségek kiszámolása a várható gólokból

3.6.2. Statisztika modell

A StatisticsModel osztály (3.7 ábra) feladata, hogy a statisztika menüben megjelenő felületi elemek számára biztosítsa a megfelelő adatokat. Az osztály az API-ból lekért adatokat dolgozza fel, rendszerezi és továbbítja a felsőbb rétegeknek. Egy példány létrehozásakor meg kell adni a figyelni kívánt ligákat, az adatbázist, valamint egy dataloader objektumot, amely az információkat szolgáltatja.

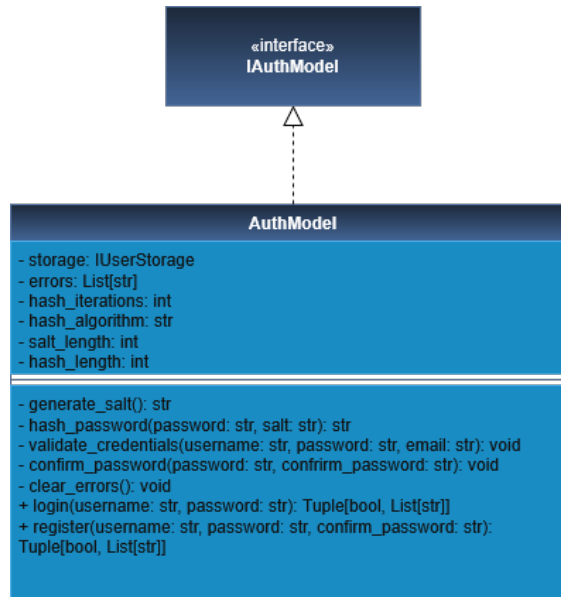


3.7. ábra. A statisztika modell osztálydiagramja

Az osztály összes metódusa az API-ból érkező adatok feldolgozására, rendezésére és továbbítására szolgál, biztosítva, hogy a statisztikai felületen megjelenő adatok naprakészek és strukturáltak legyenek.

3.6.3. Autentikáció

A mini-játékok használatához először felhasználói fiókba kell belépni, amely jelszóval védett. A belépés és a regisztráció folyamatát az AuthModel osztály (3.8 ábra) kezeli. Az osztály két publikus metódust biztosít: login és register, míg a többi, privát metódus az ellenőrzési folyamatokat végzi.



3.8. ábra. Az autentikáció modell osztálydiagramja

A jelszavakat nem szöveggént, hanem hash-elt formában tároljuk, így a modell a hash-elés és a biztonsági paraméterek alkalmazását is elvégzi. A hash-függvény és annak biztonsági paraméterei a példány létrehozásakor kerülnek megadásra, biztosítva a felhasználói adatok megfelelő védelmét.

3.6.4. Mini-játék modellek

Az alkalmazáshoz számos mini-játék készült, azonban mindegyik modelljében közös, hogy a GameModel absztrakt osztályból származik. Az absztrakt osztály biztosítja, hogy minden játéknak rendelkeznie kell egy takarító (cleanup) módszerrel, amely a játék bezárásakor felszabadítja az erőforrásokat. Továbbá a GameModel a QObject osztályból is leszármazik, lehetővé téve, hogy a modellek jelzéseket küldjenek a felhasználói felület felé.

Az egyes játékmodellek főbb feladatai a következők:

1. DribbleKingModel

- Játék elemeinek létrehozása
- Adatbázissal való kommunikáció
- Időzítő kezelése
- A játék léptetését végző `step` módszer
- Mozgás kezelése

2. DummyModel

- Placeholder modell, implementációt nem tartalmaz

3. **ReflexGameModel**

- Játék elemeinek létrehozása
- Időzítő kezelése
- Az `update` metódus rákötése az időzítőre
- Mozgások és ütközések kezelése

4. **RiskGameModel**

- Adatbázissal való kommunikáció
- Pontszámok frissítése
- Véletlenszerű események szimulálása

5. **SnakeGameModel Python**

- Adatbázissal való kommunikáció

6. **SnakeGameModel WPF**

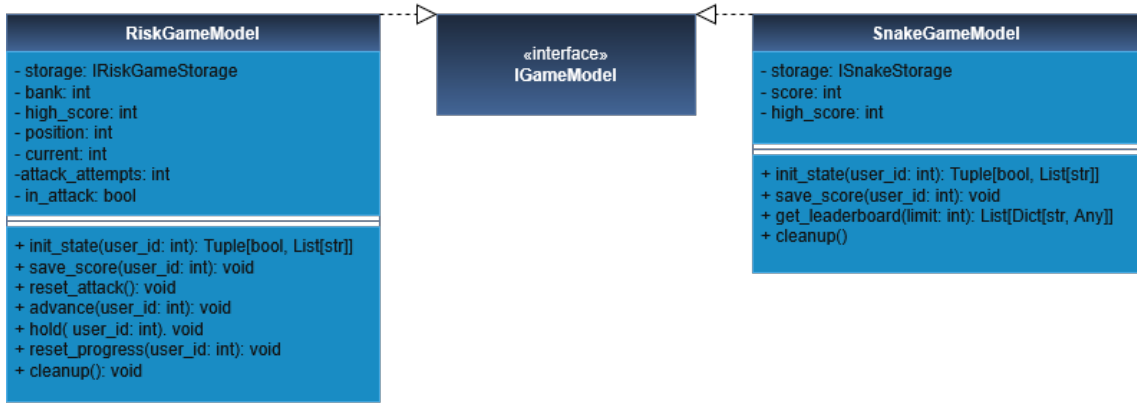
- Játék elemek létrehozása
- Időzítő kezelése és `update` metódus
- Mozgás kezelése
- A modell főként egy YouTube videó alapján készült [11]

7. **MineSweeper WF**

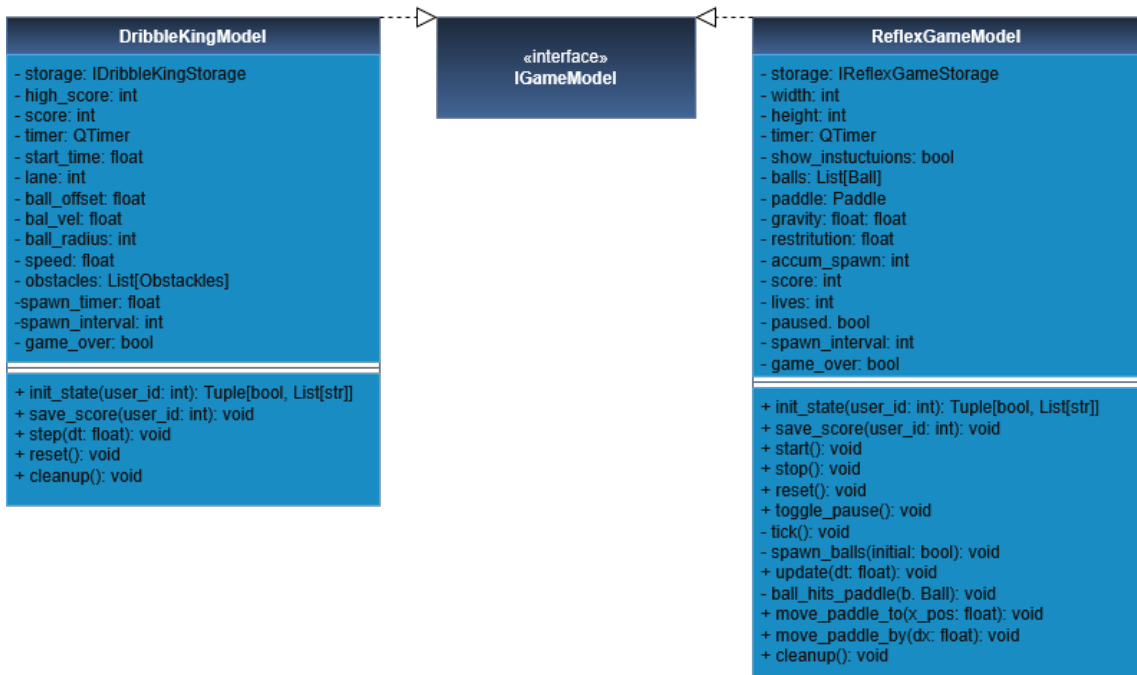
- Játék elemek létrehozása
- Rekurzió a cellák felfedésére
- Véletlen bombák elhelyezése

8. **FUTModel**

- Adatbázissal való kommunikáció és Kártyanyitás szimuláció



3.9. ábra. A kockázat és kígyó játék modelljeinek osztálydiagramjai



3.10. ábra. A reflex és cselezőkirály modelljeinek osztálydiagramjai

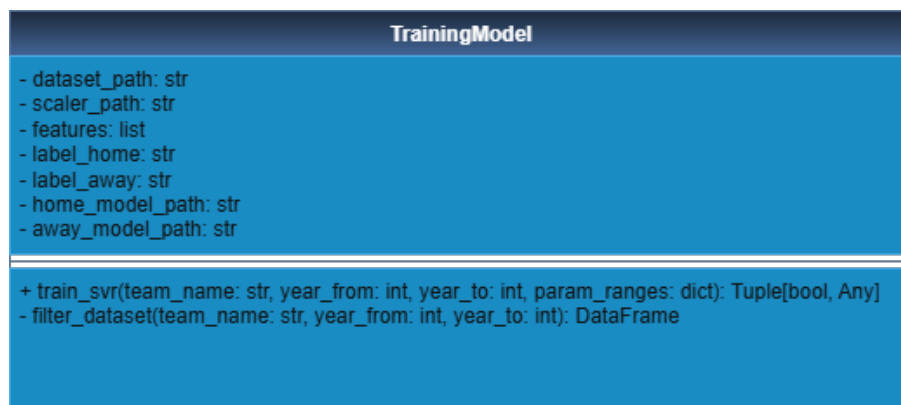
A mini-játékok funkciója az alkalmazásban kizárólag kiegészítő, könnyed kikapcsolódást biztosító szolgáltatás. A mellékelt osztálydiagram (3.9 és 3.10 ábrák) és a következők elnevezések segítik a modellek működésének és kapcsolódásainak megértését.

3.6.5. Tanító logika

A **TrainingModel** osztály (3.11 ábra) feladata egy már betanított modell további tanítása, jelen esetben a Support Vector Regressió (SVR) [3] alkalmazásával. Az osztály egy publikus metódust tartalmaz, amely a tanítási folyamat végrehajtását

koordinálja. Maga a tanítási folyamat részletes ismertetése egy későbbi szekcióban történik.

Az osztály emellett egy privát metódust is tartalmaz, amely az adathalmaz előfeldolgozásáért és szűréséért felel, biztosítva, hogy a tanításhoz használt adatok megfelelően előkészítettek legyenek.



3.11. ábra. A tanító modell osztálydiagramja

3.7. Vezérlő réteg

A vezérlő réteg feladata a modell és a nézet közötti folyamatok felügyelete és a helyes végrehajtás biztosítása. E réteg vezérlői kommunikálnak az App osztállyal is, amely az ablakváltásokat és az üzenetablakok megjelenítését kezeli. A kommunikációt a PyQt által biztosított jelzők (signals) és slotok mechanizmusa valósítja meg, lehetővé téve az eseményvezérelt adatáramlást a rétegek között.

A legtöbb vezérlő rendelkezik négy alapvető jelzővel (signal), amelyek segítségével kommunikálni tud az App példánnyal. Ezek a jelzők a következők:

- **return_to_home**: visszatérés a főoldalra.
- **switch_to_view(widget)**: váltás a paraméterben megadott nézetre.
- **error_occured(list)**: hibajelzés, a paraméterként átadott lista tartalmazza a hibaüzenetet.
- **information(list)**: információk továbbítása az üzenetablakhoz, a paraméterben átadott lista az üzeneteket tartalmazza.

Minden vezérlő rendelkezik egy cleanup metódussal, amely a memória felszabadításáért és az adatbázis-kapcsolatok megfelelő megszüntetéséért felel. Ez biztosítja

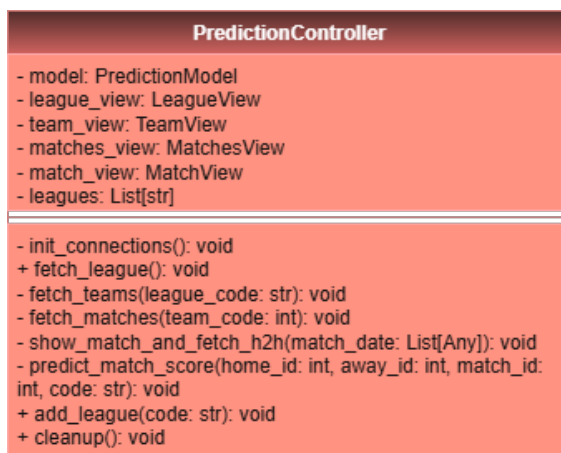
a rendszer erőforrásainak hatékony kezelését és a stabil működést a vezérlők életciklusának lezárásakor.

3.7.1. Az előrejelzések irányítása

A PredictionController osztály (3.12 ábra) biztosítja a vezérlést az előrejelző felhasználói felület és a modell között. A predikciós vezérlő létrehozásakor meg kell adni a kapcsolódó nézeteket és a modellt.

Az `init_connections` metódus felel a modell és a nézetek jelzőinek összekapcsolásáért a megfelelő kezelőmetódusokhoz. A `fetch` metódusok asszinkron módon hajtódnak végre: lekérlik az adatokat a modellből, miközben jelzést küldenek az alkalmazás példányának, hogy a felhasználó számára töltőképernyőt jelenítsen meg az adatok betöltéséig. A visszakapott adatok mellett egy boolean típusú változó is érkezik, amely a lekérés sikerességét jelzi, így a vezérlő szükség esetén hibajelzést küldhet az App osztálynak.

A nézetek frissítését az új adatokkal a `refresh` metódusok végzik, biztosítva, hogy a felhasználói felület mindig a legfrissebb információkat jelenítse meg.



3.12. ábra. Az előrejelző vezérlő osztálydiagramja

3.7.2. A statisztikák vezérlése

A StatisticsController osztály szerkezete hasonló a PredictionController-hez. A `fetch` metódusok lekérlik az adatokat a modellből, míg a `refresh` metódusok a nézetek frissítéséért felelősek.

Az adatok lekérése közben az asszinkron működés lehetőségeit kihasználva a vezérlők jelzést küldenek az alkalmazásnak, hogy a felhasználói felület töltőképernyőt

jelenítsen meg a folyamat idejére. A modellből visszakapott adatok mellett egy logikai érték is érkezik, amely a művelet sikerességét jelzi, lehetővé téve a vezérlő számára, hogy szükség esetén hibajelzést küldjön az App osztálynak.

3.7.3. A TrainingController osztály

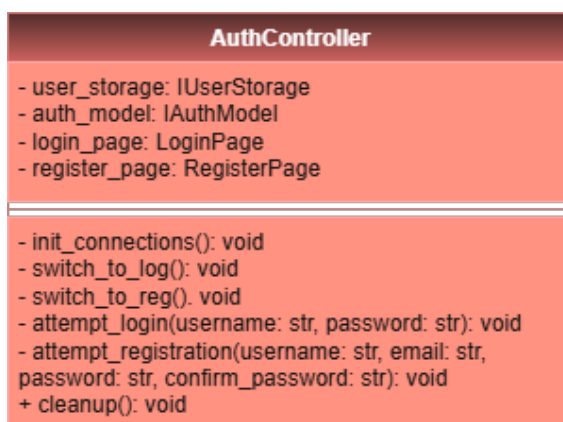
Az osztály létrehozásakor meg kell adni a tanító és a szerkesztő nézetet, valamint a modellt. A tanító nézet lehetővé teszi a tanítás folyamatának nyomon követését, míg a szerkesztő nézetben a felhasználó testreszabhatja a tanítási paramétereket.

Az `init_connections` metódus felel a jelzők és a kezelőmetódusok összekapcsolásáért. A vezérlő rendelkezik kezelőmetódusokkal a tanítás megkezdésére, befejezésére, valamint a tanítás közbeni üzenetek kezelésére, biztosítva ezzel a folyamat megfelelő vezérlését és a felhasználói felület folyamatos frissítését.

3.7.4. A mini-játékokat irányító osztályok

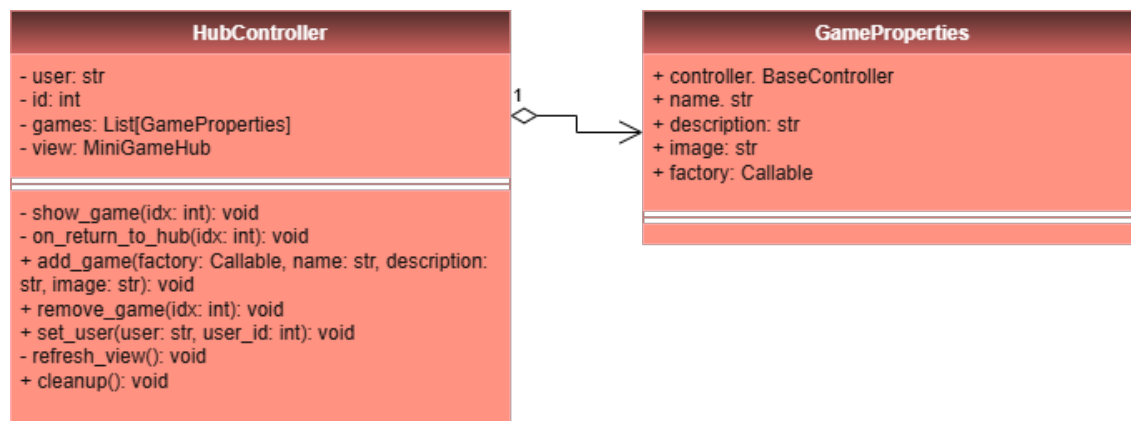
Az AuthController osztály (3.13 ábra) biztosítja az autentikáció során a nézetek és a modell közötti megfelelő kommunikációt. Az `attempt_login` és `attempt_register` metódusok felügyelik a bejelentkezés és a regisztráció folyamatát, miközben az App osztállyal kommunikálnak a hibák, üzenetek és az ablakváltás kezelése érdekében.

A `switch_to_log` és `switch_to_reg` metódusok lehetővé teszik a nézetek közötti váltást a bejelentkező és regisztrációs felület között. Sikeres bejelentkezés esetén az osztály kiadja a `login_successful` jelzést, amelynek hatására az alkalmazás automatikusan átvált a mini-játékok gyűjteményét tartalmazó nézetre.



3.13. ábra. Az autentikáció vezérlő osztálydiagramja

A HubController osztály (3.14 ábra) felelős a mini-játékok gyűjteményének kezeléséért. A játékok tulajdonságait listában tárolja, például a játék nevét, leírását, nézetét és modelljét, továbbá tartalmaz egy gyártófüggvényt a játékok felparaméterezéséhez.



3.14. ábra. A játék gyűjtemény vezérlő osztálydiagramja

4. Definíció (Gyártó). A Gyártó (a gyártófüggvény egyszerűsített változata) – Olyan objektumokat hoz létre, hogy a kliens ne lássa a példányosítás logikáját, és a létrehozott objektumra egy közös interfészen keresztül hivatkozik. [12]

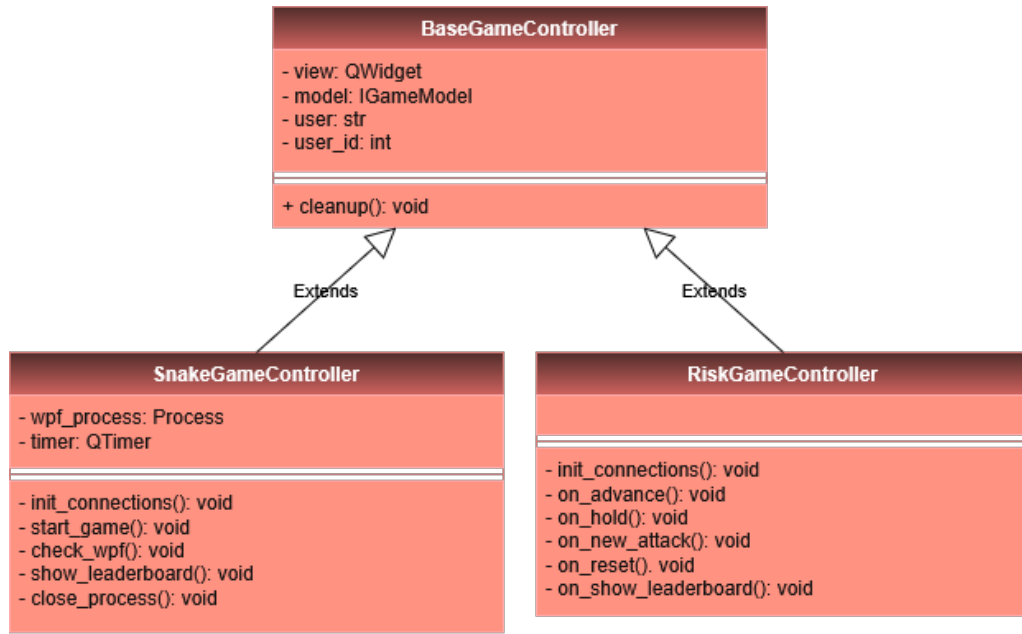
A játékok hozzáadása az `add_game` metódussal történik, amelyet az `App` osztály `login_successful` kezelője hív meg (3.4 kód), így a játékok bekerülnek a gyűjtemény listájába. A `refresh` metódus felel a nézet frissítéséért, míg a `show_game` metódus lehetővé teszi egy játék felparaméterezését és egyidejű megjelenítését a felhasználói felületen.

```

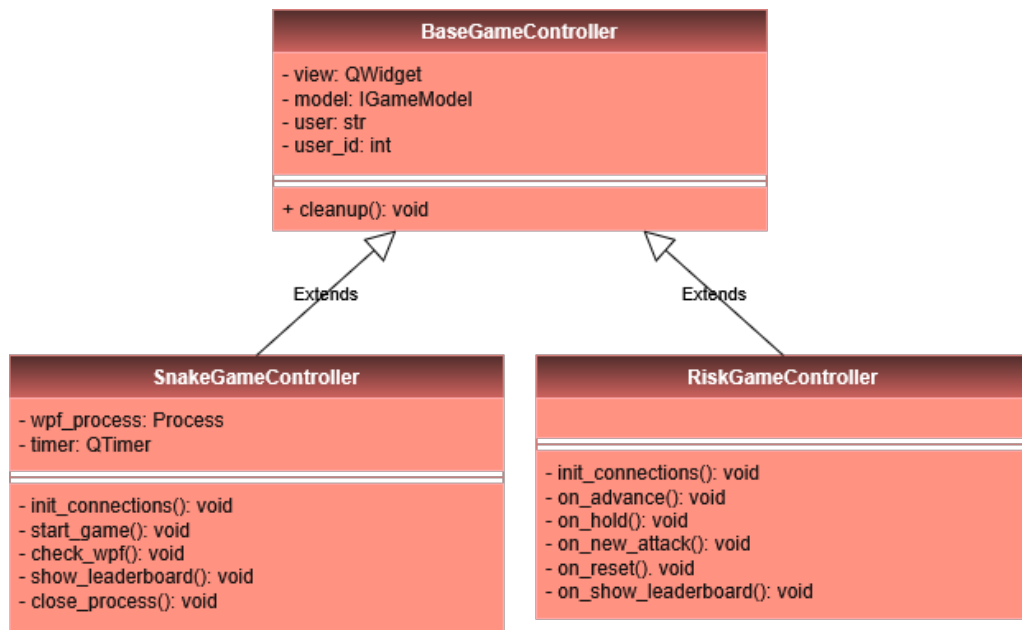
1 async def dribble_factory():
2     dribble_storage = SQLiteDribbleKingStorage("./Assets/users.db")
3     await dribble_storage.initialize_connection()
4     dribble_model = DribbleKingModel(dribble_storage)
5     await dribble_model.init_state(user_id)
6     dribble_view = DribbleKingView(dribble_model)
7     return DribbleKingController(dribble_view, dribble_model,
8                                   name, user_id)
9
10 self.hub_controller.add_game(
    dribble_factory, "Dribble King", "Dribble ball between cones",
    "./Assets/dribble.png")
  
```

3.4. forráskód. Egy játék hozzáadása gyártófüggvénnyel a játék gyűjteményhez

Egy játékból való visszatérés után az `on_return_to_hub` metódus hívódik meg, amely felszabadítja a játék által használt erőforrásokat és tárhelyet. A gyártófüggvény biztosítja, hogy a felszabadítás után a mini-játékok könnyen újra felparamétezhetők legyenek, ezáltal fenntartva a megfelelő memóriahasználatot és a rendszer hatékony működését.



3.15. ábra. Kockázat és kígyó játék vezérlők osztálydiagramjai



3.16. ábra. Cselezőkirály és reflex játék vezérlők osztálydiagramjai

Minden mini-játék vezérlője a BaseGameController osztályt (3.15 és 3.16 ábrák) valósítja meg, amely biztosítja azokat a jelzőket (signals), amelyek segítségével a vezérlők kommunikálni tudnak a HubController-rel. A mini-játék vezérlői felügyelik a játékon belüli folyamatokat, és összekapcsolják a nézet és a modell réteget a jelzők és azok kezelőmetódusai révén.

A SnakeGameController és a MineSweeperController különlegesek a játékvezérlők között (3.15 ábra), mivel külső, futtatható állományokat indítanak alfolyamatként. Ezt a folyamatot mindkét osztályban a start_game metódus biztosítja (3.5 kód), míg a check metódus figyeli az alfolyamatból érkező üzeneteket. A check metódus egy időzítőhöz is kapcsolódik, biztosítva a folyamatos ellenőrzést.

```
1 if self.__wfp_process is not None:
2     self.__close_process()
3
4 # start WPF game
5 self.__wfp_process = subprocess.Popen(
6     [resource_path(r"./Assets/SnakeGameWPF/Snake.WPF.exe")],
7     stdout=subprocess.PIPE,
8     stderr=subprocess.PIPE,
9     text=True
10 )
```

3.5. forráskód. Alfolyamat indítása

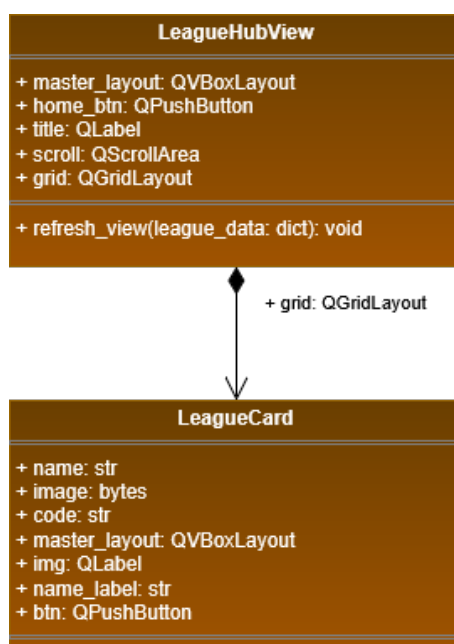
3.8. Felhasználói felület

A nézeti felület a Qt keretrendszer Python API-jára (PyQt5 [13]) épül. A PyQt keretrendszerben különféle widgetek jeleníthetők meg, amelyek a PyQt5.QtWidgets modulban találhatók. Minden megjeleníthető elem a QWidget osztályból származik, így ha egy osztály ezt örökli, akkor egyedi widgeteket is létrehozhatunk. A QtWidgets modul számos beépített elemet tartalmaz, például QPushButton, QLabel és más alapvető grafikus komponenseket. Az alkalmazás által leggyakrabban használt beépített widgeteket a (Táblázat 3.2) táblázat ismerteti. A widgetek mellett különféle layout-ok is rendelkezésre állnak, amelyek segítségével a felület elrendezése kialakítható.

Elem	Leírás
<i>QLabel</i>	Egyszerű szöveg vagy kép megjelenítésére szolgáló widget.
<i>QPushButton</i>	Kattintható gomb, események indítására (pl. művelet végrehajtása).
<i>QVBoxLayout</i>	Widgetek egymás alá rendezésére szolgáló vertikális elrendezés.
<i>QHBoxLayout</i>	Widgetek egymás mellé rendezése vízszintes elrendezésben.
<i>QTableWidget</i>	Táblázat adatok megjelenítésére; cellák szerkeszthetők és formázhatók.
<i>QScrollArea</i>	Görgethető terület, nagyobb vagy dinamikusan bővülő tartalom megjelenítésére.
<i>QTabWidget</i>	Többfüles (tabos) kezelőfelület létrehozása, oldalak közti váltással.
<i>QFormLayout</i>	Űrlapjellegű elrendezés címke–mező párokkal (pl. beviteli mezők).
<i>QPixmap</i>	Képek betöltése és kezelése (különösen megjelenítéshez QLabel-ben).
<i>QLineEdit</i>	Egy soros szövegbevitelre szolgáló beviteli mező.

3.2. táblázat. Qt widgetek és layoutok leírása

A nézetrétegben található valamennyi osztály önálló widgetet valósít meg, amely elrendezéseket és beépített komponenseket tartalmaz. Például a `LeagueHubView` osztály egy fő elrendezést definiál, amelybe `QLabel` és `QPushButton` elemek kerülnek. Az osztály szerkezetét a (3.17 ábra) diagram szemlélteti.



3.17. ábra. A bajnokságválasztó nézet osztálydiagramja

A PyQt5 csomag a QtGui és a QtCore modulokat is tartalmazza. A QtGui felel többek között a képek, videók és egyéb grafikai elemek kezeléséért (például QPixmap objektummal képek tölthetők be). A QtCore biztosítja a jelzők (signals) és slotok működését, valamint tartalmaz olyan alapvető osztályokat, mint a méretezéshez és igazításhoz szükséges konstansok (például QtCore.Qt.AlignCenter).

Minden widget rendelkezhet egyéni azonosítóval, amelyet a setName() metódussal lehet megadni. Ez a név később szelektorként szolgálhat a stíluslapok számára. A setStyleSheet() metódus segítségével CSS-szerű formátumban adhatók meg a stílusok, például: QPushButton {background-color: black;}. A stílusokat célszerű külön fájlba kiszervezni, amely .qss kiterjesztést kap. Bár ezek valójában egyszerű szövegfájlok, a megfelelő kiterjesztés és egy fejlesztői kiegészítő révén szintaxiskiemelés is használható hozzájuk. A QSS a CSS-hez hasonló szintaxist biztosít, azonban korlátozottabb funkcionalitással rendelkezik (például a transition nem támogatott). A widgetek elnevezése fontos szerepet kap, mivel QSS-ben osztály- vagy azonosítószerű megkülönböztetés helyett ezek alapján lehet a stílusokat alkalmazni. Megjegyzendő, hogy ebben a rendszerben nincs külön ID vagy class, csak az objektumnév, amely viszont több elemnél is megegyezhet.

Az aknakereső játék nézete hasonló elven épül fel, azonban nem PyQt-ben, hanem a C# Windows Forms keretrendszerben készült, annak saját grafikus elemeit használva. A kígyós játék (Snake) nézete ettől eltérően XAML deklaratív nyelven van megfogalmazva, így a felület nem egy külön nézetosztállyal, hanem XAML állományban definiáltan jön létre.

3.9. Gépi tanulós modellek tanítása

3.9.1. Adathalmaz

A tanítás megkezdése előtt szükséges olyan adatbázist választani, amely kellő terjedelmű, megbízható és nagy mennyiségű statisztikai adatot tartalmaz. Emellett lényeges szempont, hogy az adatbázis olyan információkat is tartalmazzon, amelyek API-hívásokon keresztül könnyen elérhetők. Ezen kritériumoknak kiválóan megfelel a `ashad0030` GitHub-felhasználó Match-Data adathalmaza [14], amely 2000-től egészen 2025-ig körülbelül 250 ezer mérkőzés részletes adatait tartalmazza.

A football-data.org [7] és a soccerdata [8] API-kon keresztül egyszerűen elérhetők az ELO-pontszámok, valamint az adott csapatok elmúlt öt mérkőzésének teljesítménye. A nyers adathalmaz az ELO értékeket már tartalmazza, azonban az elmúlt öt találkozó formájára vonatkozó adatok nincsenek előkészítve. Ennek feldolgozását a Refactor_matches.ipynb forrásfájl végzi.

A GitHubról CSV-formátumban letöltött adathalmaz először pandas DataFrame-be kerül beolvasásra, majd dátum szerint rendezésre. Ezt követően kiszámításra kerül mind a hazai, mind az idegenbeli csapatok elmúlt öt mérkőzésének statisztikai mutatója. A számításokat a get_last5_stats függvény végzi, amely az újonnan keletkező statisztikai jellemzők számára külön oszlopokat hoz létre. A függvény először csapat és dátum szerint rendezi az adatokat, majd csoportosítja azokat, és csúszóablakos (sliding window) aggregálást alkalmaz az elmúlt öt mérkőzés adatainak előállításához (3.6 kód).

```
1 long_df['Last5_Points'] = long_df.groupby('Team')['Points'].  
    transform(lambda x: x.shift().rolling(n, min_periods=1).sum())  
2 long_df['Last5_GoalsFor'] = long_df.groupby('Team')['GoalsFor'].  
    transform(lambda x: x.shift().rolling(n, min_periods=1).sum())  
3 long_df['Last5_GoalsAgainst'] = long_df.groupby('Team')['  
    GoalsAgainst'].transform(lambda x: x.shift().rolling(n,  
    min_periods=1).sum())  
4 long_df['Last5_Wins'] = long_df.groupby('Team')['Win'].transform(  
    lambda x: x.shift().rolling(n, min_periods=1).sum())  
5 long_df['Last5_Draws'] = long_df.groupby('Team')['Draw'].transform(  
    lambda x: x.shift().rolling(n, min_periods=1).sum())  
6 long_df['Last5_Losses'] = long_df.groupby('Team')['Loss'].transform(  
    (lambda x: x.shift().rolling(n, min_periods=1).sum())
```

3.6. forráskód. A csapatok statisztikáinak kiszámítása az elmúlt öt mérkőzés alapján

Az átalakított, kibővített adathalmaz végül CSV-formátumban kerül elmentésre, hogy a későbbi modelltanítási folyamatok során közvetlenül felhasználható legyen.

3.9.2. Tanítás

Az előrejelző modellek tanítása a Football_match_result_prediction.ipynb forrásfájlban történik. A modellek felügyelt módon kerülnek betanításra, tehát a be-

meneti adatokhoz tartozó címkék rendelkezésre állnak. A feladat regressziós jellegű, célja pedig a mérkőzések várható gólszámának becslése.

Első lépésként betöltésre kerül a korábban előfeldolgozott, kibővített CSV-adathalmaz, majd kiválasztásra kerülnek azok a tulajdonságok, amelyek a tanításhoz felhasználhatóak. A célváltozó a mérkőzés valós gólszámát tartalmazza, amelyet a modelleknek meg kell jósolniuk.

Az előfeldolgozás során megtörténik a hiányos adatok vizsgálata. Az adathalmaz nagy mérete miatt a hiányos sorok egyszerű eltávolítása nem okoz érdemi torzulást. Fontos megjegyezni, hogy az ELO pontszámok jellemzően 1000 körüli tartományban mozognak, míg a többi statisztikai jellemző általában néhány tízes nagyságrendű. Emiatt szükséges a bemeneti tulajdonságok -1 és 1 közötti intervallumra történő skálázása, amely megakadályozza, hogy a nagyobb skálájú változók dominálják a modell tanítását. A címkéket (gólszám) nem kell skálázni, mivel a cél a valós értékekhez való minél pontosabb közelítés.

A skálázást követően a különböző regressziós modellek tanítása történik. A következő modellek kerültek betanításra:

- **XGB:** Az XGBoost egy döntési fákon alapuló, gradienst követő boosting algoritmus, amely nagy hatékonysággal és teljesítménnyel rendelkezik kiterjedt adathalmazokon. A módszer iteratív módon javítja a gyenge tanulók hibáit, miközben beépített regulációs technikákat alkalmaz a túltanulás csökkentése érdekében. [15]
- **MLP:** Az MLP egy többrétegű mesterséges neurális hálózat, amely képes összetett, nemlineáris összefüggések modellezésére. A bemeneti réteget több rejtett réteg követi, így a hálózat alkalmas a bonyolult mintázatok felismerésére. Regressziós feladat esetén a kimeneti réteg egyetlen neuronból áll lineáris aktivációs függvénnel. [16]
- **Gradient Boosting:** A Gradient Boosting Regressor több sekély döntési fát illeszt egymás után úgy, hogy minden új fa a korábbiak hibáit próbálja kompenzálni. A módszer hatékony a nemlineáris összefüggések feltárásában, és kisebb, illetve közepes méretű adathalmazokon jól teljesít. Az algoritmus működési elve hasonló az XGBoostéhoz, de egyszerűbb és kevésbé optimalizált formában. [17]

- **SVR:** Az SVR a Support Vector Machine regressziós változata. Célja olyan regressziós függvény megtalálása, amely a lehető legtöbb adatpontot egy meghatározott toleranciasávon belül tart. Kernel függvények alkalmazásával az SVR képes a komplex, nemlineáris mintázatok kezelésére is, így hatékony eszköz magas dimenziójú vagy nemlineáris összefüggéseket tartalmazó adathalmazokon. [3]

A modellek tanítása során fontos a lehető legjobb hiperparaméterek keresése (3.7 kód). Erre az Optuna [18] Python csomag kínál hatékony és rugalmas megoldást, amely automatizált hiperparaméter-keresési módszereket biztosít.

```
1 def optimize_gb(X, y, n_trials=100):
2     def objective(trial):
3         params = {
4             # Core boosting parameters
5             "n_estimators": trial.suggest_int("n_estimators", 50,
6                 2000),
7             "learning_rate": trial.suggest_float("learning_rate",
8                 0.001, 0.3, log=True),
9             "subsample": trial.suggest_float("subsample", 0.5, 1.0)
10             ,
11
12             # Tree structure
13             "max_depth": trial.suggest_int("max_depth", 2, 10),
14             "min_samples_split": trial.suggest_int("
15                 min_samples_split", 2, 20),
16             "min_samples_leaf": trial.suggest_int("min_samples_leaf
17                 ", 1, 10),
18             "max_features": trial.suggest_categorical("max_features
19                 ", ["sqrt", "log2", None]),
20
21             # Loss function
22             "loss": trial.suggest_categorical("loss", ["
23                 squared_error", "absolute_error", "huber"]),
24
25             # Randomness
26             "random_state": 42,
27         }
28
29         model = GradientBoostingRegressor(**params)
30         kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
24     scores = cross_val_score(model, X, y, cv=kf,
25                               scoring="neg_mean_absolute_error",
26                               n_jobs=-1)
27
28     return np.mean(scores)
29
30
31     study = optuna.create_study(direction="maximize")
32     study.optimize(objective, n_trials=n_trials, n_jobs=-1)
33
34     print("Best params:", study.best_trial.params)
35     return study.best_trial.params
```

3.7. forráskód. A Gradient Boosting modell finomhangolása

A tanítás az átlagtól vett abszolút hibát (MAE) alkalmazza. A modellek és a tanítási folyamat kialakításának ötletét Steven McSorley GitHub-felhasználó football_prediction projektje [19] inspirálta. A szimulációs eredmények bemutatásáról az A függelékben, míg a hiperparaméterek finomhangolásáról a B függelékben lehet bővebben olvasni.

3.10. Tesztelés

3.10.1. Egységtesztelés

A modellréteg működésének ellenőrzésére fehér dobozos egységtesztek készültek. A tesztek a pytest [20] keretrendszerben kerültek megírásra, amely lehetőséget biztosít az automatizált tesztfuttatásra és az egyes osztályok viselkedésének izolált vizsgálatára.

Mivel a modellréteg függ a perzisztenciaretegből befecskendezett objektumoktól, szükségessé vált ezek szimulálása annak érdekében, hogy a tesztek kizárólag a modellréteg osztályait példányosítsák. A függőségek pótlására a unittest.mock modulban elérhető Mock-osztályok nyújtanak megoldást. A hagyományos Mock osztály ugyan képes a szinkron metódusok helyettesítésére, azonban az alkalmazás aszinkron függvényeket is használ, ezért ilyen esetekben az AsyncMock osztály alkalmazása szükséges. Ez lehetővé teszi az aszinkron műveletek valósághű szimulálását anélkül, hogy a valódi adatbázisréteg meghívásra kerülne.

A tesztelés (3.18 ábra) a projekt gyökérkönyvtárában indítható el, a parancssorban kiadott pytest utasítással (3.8 kód).

```
1 pytest -v --asyncio-mode=auto .\Test\TestAuthModel.py .\Test\
  TestFUTModel.py .\Test\TestPredictionModel.py .\Test\
  TestRiskGameModel.py .\Test\TestSnakeGameModel.py .\Test\
  TestStatisticsModel.py -W ignore::DeprecationWarning
```

3.8. forráskód. Az automatikus tesztelés indítása

A következő egységtesztek készültek el:

1. AuthModel tesztelése:

- Sikeres regisztráció
- Már létező név megadása regisztrációnál
- Érvénytelen adatok megadása regisztrációnál
- Kivételkezelés tesztelése
- Sikeres bejelentkezés
- Nem létező fiókadat megadása
- Érvénytelen adatok megadása bejelentkezésnél
- Hash függvény tesztelése

2. FUTModel tesztelése:

- Sikeres csomagnyitás
- Összes kártya ki lett nyitva már
- Kivételkezelés tesztelése
- Meglévő kártyák lekérése
- Tárhely felszabadítás

3. PredictionModel tesztelése:

- Bajnokság hozzáadása
- Ligák lekérése (sikeres)
- Ligák lekérése (sikertelen)
- Csapatok lekérése (sikeres)
- Csapatok lekérése (sikertelen)
- Mérkőzések lekérése (sikeres)
- Mérkőzések lekérése (sikertelen)
- Kivételkezelés tesztelése
- Tárhely felszabadítása

- Előrejelzés készítése

4. RiskGameModel tesztelése:

- Tárhely felszabadítása
- Kivételkezelés tesztelése
- Eredmények lekérése adatbázisból
- Tovább lépés sikeresen
- Tovább lépés sikertelenül
- Bankba menteni a pontszámot
- Kezdő pozíció felparaméterezése
- Minden statisztika törlése

5. SnakeGameModel tesztelése:

- Kivételkezelés tesztelése
- Eredmények lekérése adatbázisból
- Eredmények mentése
- Tárhely felszabadítása

6. StatisticsModel tesztelése:

- Bajnokság hozzáadása
- Adatok lekérése api-ból (sikeres)
- Adatok lekérése api-ból (sikertelen)
- Előrejelzések lekérése adatbázisból
- Tárhely felszabadítása

```
Test/TestRiskGameModel.py::test_init_state_existing_user PASSED [ 52%]
Test/TestRiskGameModel.py::test_init_state_new_user PASSED [ 54%]
Test/TestRiskGameModel.py::test_init_state_exception PASSED [ 56%]
Test/TestRiskGameModel.py::test_reset_attack PASSED [ 57%]
Test/TestRiskGameModel.py::test_advance_safe PASSED [ 59%]
Test/TestRiskGameModel.py::test_advance_tackled PASSED [ 61%]
Test/TestRiskGameModel.py::test_advance_goal PASSED [ 63%]
Test/TestRiskGameModel.py::test_hold PASSED [ 64%]
Test/TestRiskGameModel.py::test_reset_progress PASSED [ 66%]
Test/TestRiskGameModel.py::test_get_scores PASSED [ 68%]
Test/TestRiskGameModel.py::test_cleanup PASSED [ 70%]
Test/TestSnakeGameModel.py::test_init_state_new_user PASSED [ 71%]
Test/TestSnakeGameModel.py::test_init_state_existing_user PASSED [ 73%]
Test/TestSnakeGameModel.py::test_init_state_exception PASSED [ 75%]
Test/TestSnakeGameModel.py::test_save_score PASSED [ 77%]
Test/TestSnakeGameModel.py::test_get_leaderboard PASSED [ 78%]
Test/TestSnakeGameModel.py::test_cleanup PASSED [ 80%]
Test/TestStatisticsModel.py::test_add_league PASSED [ 82%]
Test/TestStatisticsModel.py::test_fetch_leagues_success PASSED [ 84%]
Test/TestStatisticsModel.py::test_fetch_leagues_rate_limit PASSED [ 85%]
Test/TestStatisticsModel.py::test_fetch_leagues_exception PASSED [ 87%]
Test/TestStatisticsModel.py::test_fetch_league_statistics_success PASSED [ 89%]
Test/TestStatisticsModel.py::test_fetch_league_statistics_exception PASSED [ 91%]
Test/TestStatisticsModel.py::test_fetch_team_statistics_success PASSED [ 92%]
Test/TestStatisticsModel.py::test_fetch_team_statistics_exception PASSED [ 94%]
Test/TestStatisticsModel.py::test_fetch_predictions_success PASSED [ 96%]
Test/TestStatisticsModel.py::test_fetch_predictions_exception PASSED [ 98%]
Test/TestStatisticsModel.py::test_cleanup PASSED [100%]
===== 57 passed in 32.77s =====
```

3.18. ábra. Egységtesztelés parancssorban

3.10.2. Manuális tesztelés

Az alkalmazás nézetrétegére fekete dobozos, manuális tesztek készültek, amelyek a felhasználói felület működését és interakcióit vizsgálták. A tesztek célja annak ellenőrzése volt, hogy a nézet megfelelően reagál-e a felhasználói műveletekre, és minden funkció a specifikációnak megfelelően jelenik meg.

A tesztek részletes leírását és eredményeit a 3.3 táblázat ismerteti.

3.3. táblázat. Manuális tesztek listája

Főoldal - /
Betöltődik a főoldal
Helyesen jelenik meg a négy gomb
Tanítás szerkesztő nézete - /
Helyesen működnek az input mezők
Gomb a bal felső sarokban a főoldalra navigál
Tanító gomb átnavigál a tanító nézetbe
Tanítás nézet - /
Szövegdobozban megjelennek a log üzenetek
Alsó gomb visszanavigál a szerkesztő nézetbe
Bejelentkező oldal - /
Input mezők helyesen működnek
Bal felső sarokban a gomb a főoldalra navigál
Alsó link a regisztrációs oldalra visz
A bejelentkezés gomb helyes adatok esetén átnavigál a mini-játékok gyűjtemény oldalra. Ellenkező esetben hiba üzenetet dob.
Regisztrációs oldal - /
Input mezők helyesen működnek
Bal felső sarokban a gomb a főoldalra navigál
Alsó link a bejelentkező oldalra visz
A regisztráció gomb helyes adatok esetén üzenetet dob fel, hogy sikeres volt a művelet. Ellenkező esetben hiba üzenetet dob fel.
Mini-játék gyűjtemény felület - /
Megjelennek betöltéskor a játék kártyák

Bal felső sarokban a gomb a főoldalra navigál
Egy kártyára történő kattintás az adott játék oldalára visz
Kockázat játék oldal - /
A ranglista gomb a ranglista nézetbe visz át
Bal felső sarokban a gomb a mini-játék gyűjtemény felületre navigál
A továbbhaladás, bankolás és törlés gomb helyesen működik
A középső szövegdobozban megjelennek a log üzenetek
Kígyó játék oldal - /
A ranglista gomb a ranglista nézetbe visz át
Bal felső sarokban a gomb a mini-játék gyűjtemény felületre navigál
Az indítás gomb elindítja a játékot egy másik ablakban
A nyilakkal lehet irányítani a kígyót
Aknakereső játék oldal - /
Az indítás gomb elindítja a játékot egy másik ablakban
Bal felső sarokban a gomb a mini-játék gyűjtemény felületre navigál
Bal egérgomb felfedi a cellát
Jobb egérgomb megjelöli a cellát
Cselezőkirály játék oldal - /
A ranglista gomb a ranglista nézetbe visz át
Jobb felső sarokban a gomb a mini-játék gyűjtemény felületre navigál
Az A/D billentyűkkel balra és jobbra lehet mozogni
Az S billentyűvel lehet elindítani a játékot
Reflex játék oldal - /
Az egérrel bal és jobbra lehet mozgatni a kesztyűt
Bal felső sarokban a gomb a mini-játék gyűjtemény felületre navigál
Kattintásra vagy bármely billentyűre elindul a játék
Statisztika kezdő oldal - /
Bal felső sarokban a gomb visszavigáz a kezdőoldalra
A két alsó gomb új oldalakra navigálnak, ahol statisztikákat lehet megtekinteni
Előrejelzések megtekintése oldal - /

Bal felső sarokban található a navigáció
Ábrákon és táblázatokon lehet megtekinteni meg az adatokat
Fül vezérlő a fülek közti váltásra
Bajnokság választó oldal - /
Bal felső sarokban található a navigáció
Kártyákon jelennek meg a bajnokságok
A ligák alatti gombra kattintva új ablakra navigál át az alkalmazás
Bajnokság oldal - /
Bal felső sarokban található a navigáció
Táblázatban látszódik a kiválasztott bajnokság tabellája, góllövő listája és az összes lejátszott mérkőzés eddig
Fül vezérlő a fülek közti váltásra
Csapatra kattintva új ablakra navigál az alkalmazás
Csapat oldal - /
Bal felső sarokban található a navigáció
Táblázatokban látszódik az eddig lejátszott mérkőzések, csapat statisztikák és játékosok listája
Fül vezérlő a fülek közti váltásra
Csapatválasztó oldal - /
Bal felső sarokban található a navigáció
Kártyákon látszódnak a csapatok a kiválasztott ligából
Kártyákon a gomb új oldalra navigál át
Mérkőzés választó oldal - /
Bal felső sarokban található a navigáció
Kártyákon, illetve naptár nézetben látszódnak a mérkőzések
Fül vezérlő a fülek közti váltásra
Kártyákon lévő gomb, illetve a mérkőzés napjára kattintás új oldalra navigál
Előrejelzés készítése - /
Bal felső sarokban található a navigáció
Modell választó gombsor az oldal tetején, alatta a mérkőzés adatok

Gomb a predikció készítéséhez. Ha nincs kiválasztva modell, akkor hiba üzenet. Ha van kiválasztva modell, akkor alul megjelennek az eredmények.

Fül vezérlő az egymás elleni mérkőzések és a predikciók közötti váltásra
--

3.11. Továbbfejlesztési lehetőségek

Az alkalmazás életciklusának jelentős részét nem a kezdeti fejlesztési fázis, hanem a hosszú távú karbantartás és folyamatos továbbfejlesztés teszi ki. Ennek megfelelően számos olyan irány létezik, amelyben a rendszer tovább bővíthető, optimalizálható vagy új funkciókkal gazdagítható. A jelenlegi implementáció stabil alapot biztosít, azonban a továbblépés érdekében több fejlesztési lehetőség is megfogalmazható.

- 1. Platformfüggetlen működés biztosítása:** A rendszer jelenleg kizárólag Windows operációs rendszer alatt érhető el, ami jelentősen korlátozza a felhasználói bázist. A többplatformos támogatás – például Linux, macOS vagy akár mobil operációs rendszerek (Android, iOS) – bevezetése növelné a hozzáférhetőséget és a rendszer rugalmasságát.
- 2. A réteges architektúra webes továbbfejlesztési potenciálja:** Az alkalmazás jól strukturált, a modell-, perzisztencia- és vezérlőréteg világosan elkülönül. Ez a szoftverarchitektúra kiváló alapot biztosít egy webes verzió kialakításához. A backend komponensek kisebb módosításokkal szolgáltatásorientált architektúrába illeszthetők, míg a frontend egy modern webes technológiával (pl. React, Angular, Vue) valósítható meg.
- 3. Nagyobb és minőségibb adathalmaz bevonása a modell tanításába:** A jelenlegi fejlesztési keret kizárólag ingyenesen elérhető adatokra és eszközökre támaszkodik. Bár ezek megfelelőek egy prototípus kialakításához, a modell teljesítménye jelentősen növelhető lenne nagyobb mennyiségű, strukturáltabb és szakmailag ellenőrzött adathalmazok bevonásával. Fizetős adatforrások, professzionális annotációs szolgáltatások vagy célzott adatgyűjtés alkalmazása a modell pontosságát és megbízhatóságát is javítaná.

4. **További mini-játékok implementálása:** Az alkalmazás jelenlegi játékmechanikái bővíthetők új mini-játékokkal, amelyek révén a felhasználók többféle módon interakcióba léphetnek a rendszerrel.
5. **Új bajnokságok integrálása** A jelenlegi alkalmazás bizonyos bajnokságokra korlátozódik, azonban a támogatott ligák körének bővítése jelentősen megnövelné a rendszer által lefedett sportági ökoszisztémát.

4. fejezet

Összegzés

A szakdolgozatomban egy asztali alkalmazás fejlesztését valósítottam meg, amely különböző gépi tanulási modellek segítségével képes előrejelezni a labdarúgó-mérkőzések eredményeit. A felhasználói dokumentáció célja, hogy az átlagos felhasználó számára is érthető módon mutassa be az alkalmazás működését, illusztrációkkal segítve a folyamatok megértését.

A fejlesztői dokumentáció részletesen bemutatja az alkalmazás összes architektúrális rétegét, kiegészítve a gépi tanulási modellek tanítási folyamatának leírásával is. A fejlesztés során alkalmaztam a gépi tanulás, az adatbázis-kezelés és az eseményvezérelt alkalmazások területén szerzett ismereteimet. Emellett lehetőségem nyílt egy számomra korábban ismeretlen könyvtár megismerésére és alkalmazására, melynek használatában kezdeti tapasztalatokkal rendelkeztem.

A projekt során az eddigi legnagyobb fejlesztésemet valósítottam meg, amely lehetővé tette, hogy a programozási ismereteimet személyes érdeklődési körömmel, a labdarúgással ötvözzem. Személyes tapasztalatom alapján a jelenlegi alkalmazás stabil alapot biztosít egy későbbi, továbbfejlesztett modell létrehozásához, amely már anyagi ráfordítással készülhet, míg a jelenlegi projekt teljes egészében ingyenesen valósult meg.

Köszönetnyilvánítás

Szeretnék köszönetet mondani témavezetőmnek, Varga Dánielnek, aki segítségével hozzájárult a szakdolgozatom elkészítéséhez. Hálával tartozom továbbá a kar oktatóinak a tanulmányaim során megszerzett tudásért, valamint azért, hogy előadásaikon és gyakorlatain részt vehettem.

A. függelék

Tanítási eredmények

A modellek mindegyike 1 alatti MAE értéket ért el, ami viszonylag kis adathalmaz (250 ezer mérkőzés nem számít soknak) és korlátozott számítási erőforrások mellett elfogadható teljesítménynek számít. Az, hogy a MAE érték 1 alatt van, azt jelzi, hogy a modellek pontosabbak a hagyományos, mindig az 1–1-es eredményt tippelő modelleknél.

Modell	Hazai MAE	Idegenbeli MAE
<i>XGB</i>	0.9438	0.8189
<i>MLP</i>	0.9510	0.8234
<i>Gradient Boosting</i>	0.9133	0.7975
<i>SVR</i>	0.9302	0.8143

A.1. táblázat. A tanítás eredményei

B. függelék

Hiperparaméter finomhangolás

A hiperparaméterek finomhangolása az `Optuna` csomag [18] segítségével valósult meg. Az `Optuna` a hiperparaméterek optimalizálását automatizált, intelligens keresési módszerekkel végzi, többek között Bayes-optimalizációs eljárásokkal. Az `Optuna`-ban belül egy *study* objektumot kell létrehozni, amely számára meg kell adni egy függvényt (objective function). Ez hordozza a hiperparaméter-teret, valamint tartalmazza a modell tanítási és értékelési lépéseit. A tanítási folyamatot az úgynevezett *trialok* alkotják, amelyek során az `Optuna` a beépített keresési algoritmusok alapján, a korábbi eredmények figyelembevételével választ új paraméterkombinációkat.

A finomhangolás során az adathalmaz a *KFold* technikával szét lett bontva. Ennek célja, hogy kiküszöbölje az adathalmaz különböző részei közötti esetleges eltéréseket, és robusztusabb teljesítménymérést biztosítson.

B.1. A finomhangolt hiperparaméterek

A finomhangolás három modell esetében történt: XGBoost, Gradient Boosting és Multi-Layer Perceptron. Az alábbiakban bemutatom a vizsgált hiperparamétereket, valamint röviden indoklom szerepüket a modell működésében.

B.1.1. XGBoost

A főbb hiperparaméterek szerepe a következő:

- **n_estimators**: a boosting körök száma; magasabb érték növelheti a modell pontosságát, de túltanuláshoz vezethet.

- **learning_rate**: a tanulási lépés nagysága; logaritmikus skálán keresett értéke stabilabb konvergenciát biztosít.
- **max_depth**, **min_child_weight**: a fák komplexitását szabályozó paraméterek, amelyek fontosak a túlilleszkedés kontrollálásában.
- **subsample**, **colsample_bytree**: rész-mintavételezési arányok, amelyek csökkentik a varianciát és javítják a generalizációt.
- **reg_alpha**, **reg_lambda**: L1 és L2 regulációs tagok; a modell robusztusságát növelik.

B.1.2. Gradient Boosting Regressor

A gradient boostingnál a következő paraméterek bírnak nagy jelentőséggel:

- **max_depth**: meghatározza az egyes döntési fák komplexitását.
- **min_samples_split** és **min_samples_leaf**: a túlilleszkedést kontrolláló strukturális paraméterek.
- **loss**: különböző veszteségfüggvények, melyek érzékenyek lehetnek az outlierekre (pl. a Huber-veszteség).

B.1.3. Multi-Layer Perceptron

A legfontosabb paraméterek:

- **hidden_layer_sizes**: a háló architektúrája, amely meghatározza a modell kapacitását.
- **activation**: a rejtett rétegek aktivációs függvénye; eltérő nemlinearitásokat ad.
- **solver**: az optimalizálási eljárás; nagyobb hálónál az **adam** jellemzően stabilabb.
- **alpha**: L2 reguláció mértéke, amely a túlilleszkedés ellen hat.
- **learning_rate_init**: a kezdeti tanulási ráta, amely erősen meghatározza a konvergencia sebességét.

C. függelék

A mesterséges intelligencia használatának nyilatkozata

A dolgozat írása és a fejlesztés során mesterséges intelligencia alapú eszközt (OpenAI ChatGPT) az alábbi célokra használtam:

- szövegek nyelvtani és stilisztikai ellenőrzése,
- kódrészletek gyors hibakeresése és általános technikai magyarázatok kérése.
- alkalmazás logójának generálása

Az alkalmazás tervezése, fejlesztése, működési logikája és a dokumentáció érdemi tartalma saját munka.

Irodalomjegyzék

- [1] *Laws of the Game / IFAB*. URL: <https://www.theifab.com/> (elérés dátuma 2025. 11. 14.).
- [2] *The history of football analytics - Part 3*. URL: <https://xfbanalytics.com/blog/blog-post/3> (elérés dátuma 2025. 11. 14.).
- [3] *Support Vector Regression (SVR) Tutorial for Machine Learning*. URL: <https://www.analyticsvidhya.com/blog/2020/03/support-vector-regression-tutorial-for-machine-learning/> (elérés dátuma 2025. 11. 15.).
- [4] *Qt Documentation*. URL: <https://doc.qt.io/> (elérés dátuma 2025. 11. 19.).
- [5] Roberto Giachetta. *Szoftver architektúrák alapvetései*. Oldal: 12. URL: https://people.inf.elte.hu/groberto/elte_kasz/eloadas_anyagok/elte_kasz_ea08_dia.pdf (elérés dátuma 2025. 11. 21.).
- [6] Roberto Giachetta. *Szoftver architektúrák alapvetései*. Oldal: 21. URL: https://people.inf.elte.hu/groberto/elte_kasz/eloadas_anyagok/elte_kasz_ea08_dia.pdf (elérés dátuma 2025. 11. 21.).
- [7] *football-data.org API Quickstart Guide*. URL: <https://www.football-data.org/documentation/quickstart> (elérés dátuma 2025. 11. 19.).
- [8] *soccerdata: Club Elo Module Documentation*. URL: <https://soccerdata.readthedocs.io/en/latest/reference/clubelo.html> (elérés dátuma 2025. 11. 19.).
- [9] *Poisson distribution*. URL: https://en.wikipedia.org/wiki/Poisson_distribution (elérés dátuma 2025. 11. 21.).

- [10] Konstantinos Loukas és tsai. „Predicting Football Match Results Using a Poisson Regression Model”. *Applied Sciences* 14.16 (2024), 7230. old. DOI: 10.3390/app14167230. URL: <https://www.mdpi.com/2076-3417/14/16/7230> (elérés dátuma 2025. 11. 21.).
- [11] *Programming a Snake Game in C# - Full Guide*. URL: <https://www.youtube.com/watch?v=uzAXxFBbVoE> (elérés dátuma 2025. 11. 23.).
- [12] *Factory design pattern*. URL: <https://www.oodeesign.com/> (elérés dátuma 2025. 11. 21.).
- [13] *The complete PyQt5 tutorial*. URL: <https://www.pythonguis.com/pyqt5-tutorial/> (elérés dátuma 2025. 11. 22.).
- [14] *Club Football Match Data (2000 - 2025)*. URL: <https://github.com/ashhad0030/Match-Data?tab=readme-ov-file> (elérés dátuma 2025. 11. 22.).
- [15] *XGBoost*. URL: <https://www.nvidia.com/en-us/glossary/xgboost/> (elérés dátuma 2025. 11. 22.).
- [16] *MLP*. URL: <https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning> (elérés dátuma 2025. 11. 22.).
- [17] *Gradient Boosting*. URL: <https://www.geeksforgeeks.org/machine-learning/ml-gradient-boosting/> (elérés dátuma 2025. 11. 22.).
- [18] *Optuna*. URL: <https://optuna.org/> (elérés dátuma 2025. 11. 22.).
- [19] *Football Match Prediction*. URL: https://github.com/stevenmcsorley/football_prediction (elérés dátuma 2025. 11. 22.).
- [20] *Pytest Documentation*. URL: <https://docs.pytest.org/en/stable/> (elérés dátuma 2025. 11. 23.).

Ábrák jegyzéke

2.1. xGoalPro telepítés	9
2.2. xGoalPro főoldal	9
2.3. Liga választás	11
2.4. Csapat választás előrejelzéshez	11
2.5. Mérkőzés választás előrejelzéshez	12
2.6. Mérkőzések naptár nézetben	12
2.7. Előrejelző ablak részlet	13
2.8. Előrejelzés eredmény	13
2.9. Egymás elleni mérkőzések	14
2.10. Statisztika kezdőoldal	14
2.11. Predikció ábrák	15
2.12. Előrejelzések táblázat részlet	16
2.13. Bajnokság statisztikák részlet	17
2.14. Csapat statisztikák részlet	17
2.15. Bejelentkezés	18
2.16. Regisztráció	19
2.17. Mini-játékok gyűjtemény	20
2.18. Csomagnyitó játék	20
2.19. Kockázat játék ranglista	22
2.20. Kígyó játék kezdőoldal	22
2.21. Kígyó játék ranglista	23
2.22. Kígyó játék felület	23
2.23. Kígyó játék használati eset diagram	24
2.24. Aknakereső kezdőoldal	24
2.25. Aknakereső felület	25
2.26. Aknakereső használati eset diagram	25
2.27. Cselezőkirály játék	26

2.28. Reflex játék	27
2.29. Paraméter szerkesztő ablak részlet	28
2.30. Tanító ablak részlet	29
2.31. Hiba ablak	30
3.1. Visual Studio komponens telepítés	34
3.2. Az alkalmazás csomagdiagramja	37
3.3. App osztálydiagramja	39
3.4. Részleges perzisztencia osztálydiagram: api hívások	43
3.5. Részleges perzisztencia osztálydiagram: előrejelzés adatbázis	43
3.6. A predikciós modell osztálydiagramja	45
3.7. A statisztika modell osztálydiagramja	47
3.8. Az autentikáció modell osztálydiagramja	48
3.9. A kockázat és kígyó játék modelljeinek osztálydiagramjai	50
3.10. A reflex és cselezőkirály modelljeinek osztálydiagramjai	50
3.11. A tanító modell osztálydiagramja	51
3.12. Az előrejelző vezérlő osztálydiagramja	52
3.13. Az autentikáció vezérlő osztálydiagramja	53
3.14. A játék gyűjtemény vezérlő osztálydiagramja	54
3.15. Kockázat és kígyó játék vezérlők osztálydiagramjai	55
3.16. Cselezőkirály és reflex játék vezérlők osztálydiagramjai	55
3.17. A bajnokságválasztó nézet osztálydiagramja	57
3.18. Egységtesztelés parancssorban	64

Táblázatok jegyzéke

2.1. Regisztrációs követelmények	19
2.2. A Kockázat játék gombsorának leírása	21
2.3. Svr tanítás paramétereit	28
3.1. Python csomagok leírása	35
3.2. Qt widgetek és layoutok leírása	57
3.3. Manuális tesztek listája	65
A.1. A tanítás eredményei	72

Forráskódjegyzék

3.1. A szükséges Python csomagok telepítése	34
3.2. A main.py, az alkalmazás belépési pontja	38
3.3. A valószínűségek kiszámolása a várható gólokból	46
3.4. Egy játék hozzáadása gyártófüggvénnyel a játék gyűjteményhez	54
3.5. Alfolyamat indítása	56
3.6. A csapatok statisztikáinak kiszámítása az elmúlt öt mérkőzés alapján	59
3.7. A Gradient Boosting modell finomhangolása	61
3.8. Az automatikus tesztelés indítása	63